

VEDA (Vehicle Evaluation Diagnostics Agent): Enables proactive Vehicles Maintenance by Predictive Failures and Optimizing Fleet Readiness using AI-Driven Analytics

B.Tech-CE, Semester-VI

Prepared at



**Bhaskaracharya National Institute for Space Applications & Geo-informatics
Ministry of Electronics and Information Technology, Govt. of India.
Gandhinagar**

Prepared By

Adhish Prasad

ID No. 26BISAG0542

Barbhaya Harsh

ID No. 26BISAG0543

Yash Verma

ID No. 26BISAG0544

Guided By:

Dr. Zalak Thakker

SCT

IAR, Gandhinagar.

External Guide:

Viraj Choksi

Project Manager,

BISAG-N, Gandhinagar

SUBMITTED TO

Institute of Advanced Research-Gandhinagar



Institute of Advanced Research - Gandhinagar

**Bhaskaracharya National Institute for Space Applications
and Geo-informatics**

MeitY, Government of India

**प्रमाणपत्र
CERTIFICATE**

दिनांक/Date: _____

यह प्रमाणित किया जाता है कि **इंस्टिट्यूट ऑफ एडवांस्ड रिसर्च, गांधीनगर** के **छठा सेमेस्टर बिटेक -सीई** के छात्र **श्री अधीश प्रसाद, श्री हर्ष बारभाया और श्री यश वर्मा** ने बायसेग-एन में छठा सेमेस्टर की समर इंटरशिप सफलतापूर्वक पूरी कर ली है। उन्होंने **17 जून, 2026 से 16 सितंबर, 2026 तक वेदा (व्हीकल इवेल्यूएशन डायग्नोस्टिक एजेंट्स): इनेबल प्रेक्टिस व्हीकलस् मैटेनन्स बाइ प्रीडिक्टिव फैलईयर एण्ड ऑप्टिमाइज़िंग फ्लीट रेडिनेस युसिंग ऐआई - ड्रीवन एनालिसिस के विकास** पर काम किया है।

जहाँ तक हमें सूचना है, यह उनके द्वारा किया गया एक मौलिक और प्रामाणिक कार्य है। संस्थान में अपने कार्यकाल के दौरान, उन्होंने आत्म-प्रेरणा और कर्तव्यनिष्ठा का परिचय देते हुए काम किया। हम उनके उज्ज्वल भविष्य की कामना करते हैं।

This is to certify Mr. Adhish Prasad, Mr. Harsh Barbhaya and Mr. Yash Verma, students of 6th Semester B.Tech-CE from Institute of Advanced Research, Gandhinagar, have satisfactorily completed the sixth semester summer internship at BISAG-N. They have worked on VEDA (Vehicle Evaluation Diagnostics Agent): Enables proactive Vehicles Maintenance by Predictive Failures and Optimizing Fleet Readiness using AI-Driven Analytics, from June 17th, 2026 to September 16th, 2026.

To the best of our information this is an original and bonafide work done by them. During their tenure at the institute, they demonstrated self-motivation and conscientiousness. We wish them a successful future ahead.

विराज चोकसी
Viraj Choksi
परियोजना प्रबंधक
Project Manager,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

पुनीत लालवानी
Punit Lalwani
अपर निदेशक सह सीआईएसओ
Additional Director cum CISO,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

CERTIFICATE

This is to certify that the 6th Semester Summer Internship Project entitled "VEDA (Vehicle Evaluation Diagnostics Agent): Enables proactive Vehicles Maintenance by Predictive Failures and Optimizing Fleet Readiness using AI-Driven Analytics" has been carried out by Adhish Prasad, Harsh Barbhaya & Yash Verma under my guidance in fulfilment of the degree of Bachelor of Technology in COMPUTER ENGINEERING (6th Semester) of Institute of Advanced Research during the academic year 2023-2027

Guide

Dr. Zalak Thakker

Head of the Department

Dr. Sanjay Sonar



About BISAG-N



Modern day planning for inclusive development and growth calls for transparent, efficient, effective, responsive and low-cost decision-making systems involving multi-disciplinary information such that it not only encourages people's participation, ensuring equitable development but also takes into account the sustainability of natural resources.

The applications of space technology and Geo-informatics have contributed significantly towards the socio-economic development. Taking cognizance of the need of geo-spatial information for developmental planning and management of resources, the department of Ministry of Electronics and Information Technology, Government of India, established "Bhaskaracharya National Institute for Space Applications and Geo-informatics" (BISAG-N).

BISAG-N is ISO 9001, ISO 27001, ISO 19115, ISO 19157 and CMMI: L5 certified national institute. BISAG-N, which was initially set up to carry out space technology applications, has evolved into a center of excellence, where research and innovations are combined with the requirements of users and thus acts as a value-added service provider, a technology developer and as a facilitator for providing direct benefits of space technologies to the grass root level functions/functionaries.



BISAG- N Journey

The journey of Institute started as a state government organization, named as **Remote Sensing and Communication Centre (RESECO)** operating under a limited mandate from 1997 to 2003, focusing on utilization of space technologies such as satellite communication, remote sensing and geospatial applications for supporting various developmental initiatives exclusive for the State of Gujarat. Opinions of various stakeholders at grass root levels was obtained to ensure alignment of Institute's work with the development and welfare need of the society. Numerous sessions of different strata of society were organized including Member of Legislative Assembly (MLAs), village sarpanch, farmers, various skill workers, women and students to obtain insights into what are the needs with simplicity of solutions, and processes of their groups and how technology can fulfil those needs and alleviate any constraints for improving the quality of life.

The basic works of maps generation as per the needs of various state government departments laid the groundwork for the institute's advancement. In December 2003, the name of the institute changed as **BISAG (Bhaskaracharya Institute for Space Applications and Geo-informatics)** with an enhanced mandate to promote satellite communication-based education and skill development facility thereby ensuring increased and more actively involvement in governance processes. The name of the institute was changed to honor the celebrated 12th-century mathematician and astronomer, Bhaskaracharya (1114-1185 AD) who provided in-depth insights about various astronomical and mathematical concepts, formulae and calculations.

The institute's services were extensively utilized by various departments of Gujarat Government under the leadership and patronage of the then Honorable Chief Minister Shri Narendrabhai Modi. Numerous times the assembly meetings were held in BISAG-N premises to amalgamate governance and technology for the well-being of the state. The state leadership expressed to the ministers and bureaucrats on multiple occasions the importance of visualization of development plans and schemes on maps and thereby harnessing the capabilities of BISAG as a **"go-to place"** for various technological and geo-informatics solutions required by concerned ministries and departments for the benefit of citizens and development of the state.

On one of such occasions in 2004, Honorable Chief Minister Shri Narendra Modi said, "Friends, I can clearly visualize that within a span of five years, all streams of government administration shall converge into BISAG. This institute shall be a huge treasure of information, if any decisions are to be taken or any department require any information or need any policy change; the **'Source of Information'** shall be BISAG. There for earlier we can transform BISAG into a **'Source of Information'** and position it as back-bone of all governance and administration activities, and the sooner the benefits of science and technology shall be available to the society."



Since then, inspired by above and gaining insights while working collaboratively for various development initiatives of central and state government department, the Institute is focused on following vision:

"To empower and serve the nation through development of space and geo-spatial technologies, its applications, solutions and services in accordance to governance principles for socio-economic welfare of the society."

The Institute develops geo-spatial technologies, databases, software and processes along with skilled resources to provide visualization, analysis and decision support system to various state government

departments. Also, the Satellite Communication infrastructure was enhanced to add more channels and developed in house expertise to install, operate and maintain various electronic instruments and equipment. The institute constantly ensured adoption of national and international standards as well as best practices to develop and reinforce the solutions delivery process. With the improved confidence and trust of the state government to utilize geo-spatial and satellite communication services for developmental, social welfare and disaster management areas, the institute was supported by the Government of Gujarat to construct dedicated buildings for ensuring services are provided at even larger scale. This expansion broadened the institute's scope to integrate advanced techniques such as managing large databases, mathematical modelling, enabling prediction and simplified architecture for the solution aligning with the government priorities for improving efficiency, transparency and pace of development

Along with providing the e-governance support to Gujarat State, the institute also started supporting various other states and central government ministries as and when required for map and decision support system development services. Recognizing this potential, the institute's mandate was further enhanced to cater to the geo-spatial and satellite communication-based e-governance services to all central government ministries as well as various states and union territories as a **National** institute. The name of the institute was then changed as Bhaskaracharya **National** Institute for Space Applications and Geo-informatics (**BISAG-N**), registered as an Autonomous Scientific Society, as per the Societies Registration Act, 1860, under the Ministry of Electronics and Information Technology (MeitY), Government of India (GoI). BISAG-N; enhanced mandate includes to undertake technology development & management; research & development; facilitate national and international cooperation and capacity building; support technology transfer and entrepreneurship development in area of geo-spatial technology.

Today, BISAG-N is collaboratively working with 50+ central government ministries and most of the states and union territories for supporting various developmental programs and citizen centric services. BISAG-N provides the technological support to these stakeholders from its Gandhinagar and New Delhi facilities. These facilities have adequate skilled personnel and state-of-the-art infrastructure for providing geo-spatial solutions and software applications including support of satellite communication-based education and training. Thus, BISAG-N is playing pivotal role as a one stop center for providing geo-spatial and space technologies for various flagship programme being undertaken for the development of nation in line with geo-spatial education, logistics and various other policies and program of central and state governments.



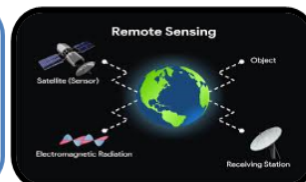


Satellite Communication:

For promotion and facilitation of the use of broadcasting facility for distant interactive training, education, skill development, health, agriculture and extension

Remote Sensing:

For inventory, mapping, developmental planning and monitoring of natural and built resources

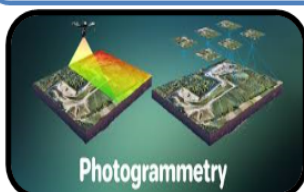


Geographic Information System:

For conceptualization, creation and organization of multi-purpose common digital databases for development of sectoral/integrated decision systems.

Global Navigation Satellite System:

For location based services, engineering applications and research

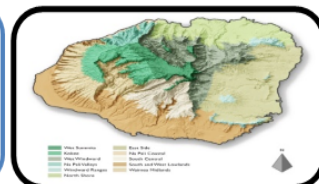


Photogrammetry:

For creation of maps, Digital Elevation Model (DEM), terrain characteristic maps for resource planning and management

Cartography:

For Thematic Mapping and generation of value added maps



Software Applications Development:

For wider usage in geo-spatial applications, Decision Support Systems (DSS) and ERP solutions.

Innovation, Research & Development:

Quantum Computing, Digital Twins, Cyber Security, Block Chain, Big Data, AI/ML

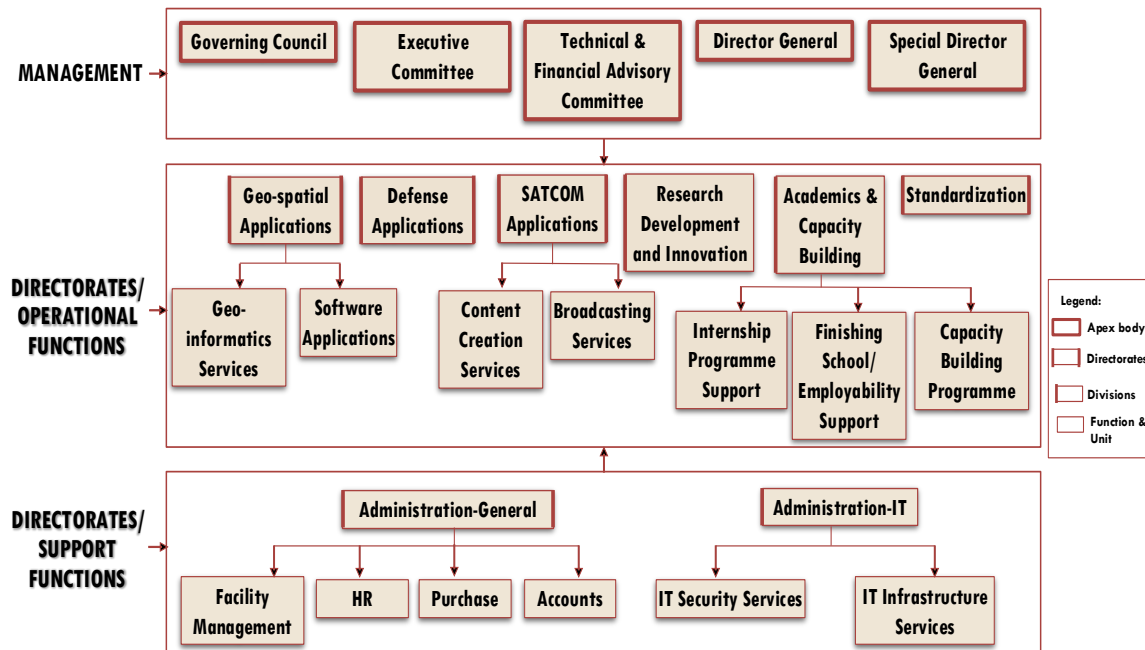


Education and Training:

For providing Education, Training & Technology knowledge transfer to large number of students, end users and collaborators for promoting Research & Development

Organizational Setup

BISAG-N main domain areas include Satellite Communication, Geo-spatial Solutions and Technology Development, Geo-informatics Applications Development, Standardization, Academics & National/ International Co-operation and Administration.



BISAG-N: Functions and Divisions

SATCOM Applications

The SATCOM Directorate provides services of distance education, capacity building, training, extension, awareness and empowerment using the state-of-the-art facility comprises of an uplink earth station, control room, studios and a satellite communication-based network. The services are primarily utilized by Ministry of Education and various other central and states ministries. Through Ministry of External Affairs (MoEA), BISAG-N is now poised to provide SATCOM based education to neighbouring countries also.

SATCOM operations are inspired by the wisdom of Hon'ble Prime Minister of India as elicited on numerous occasions. For example, in 2004 as Hon'ble Chief Minister of Gujarat, Shri Modi explained the importance of learning through the ancient way of visualization using virtual media with these words:

“The way Eklavya studied by making a statue, why we can't study with the help of a T.V.? Yes, we can. Was any cinema song ever taught to us in any class? Was it ever taught by any teacher? We learnt song from T.V., isn't it? If we have learnt entire cinema song, learnt every word, learnt scale, learnt rhythm, it means we can learn through T.V. also.”



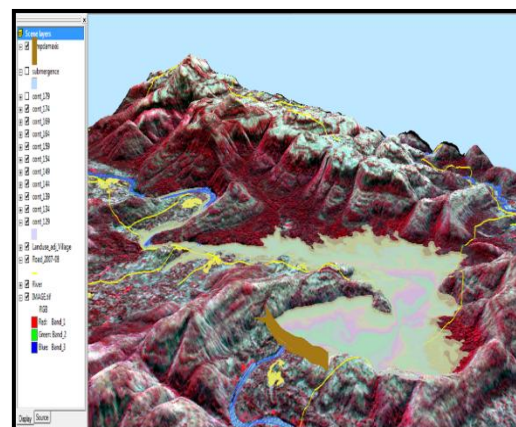
There are twelve state-of-the-art studios for live recording of content supported by dedicated editing labs, subtitling, translation, animation and various other aspects. The institute has set up ten studios at Gandhinagar facility and another two studios at New Delhi. Infrastructure for transmission and Ku band uplink teleports (11m/9.3m) of more than 350 channels is installed, integrated and maintained by inhouse team. BISAG-N support various initiatives of Ministry of Education, Government of India, as well as other central and state government ministries such as PM e-Vidya, Swayam Prabha etc. as detailed further.



Geo-spatial Applications Directorate

BISAG-N's Geo-Spatial Applications Directorate provides specialized services and solutions to implement Geographic Information Systems (GIS) for providing impetus to planning and developmental activities at grass root level. The objective of GA directorate is to provide analysis and decision support services to various stakeholders through scientifically organized, comprehensive, multi-purpose, compatible and large scale geo-spatial databases.

Geo-spatial Applications Directorate creates and organizes large-scale and multi-purpose geospatial datasets in collaboration with central and state government departments and agencies. A well-qualified team of GIS technicians and engineers ensure that the datasets are compatible, standardized and authenticated for use in analysis, planning and decision-support functions. Goals and roles of converging organizations are taken into consideration while geo-spatial database creation and software development activities



Software Development Division

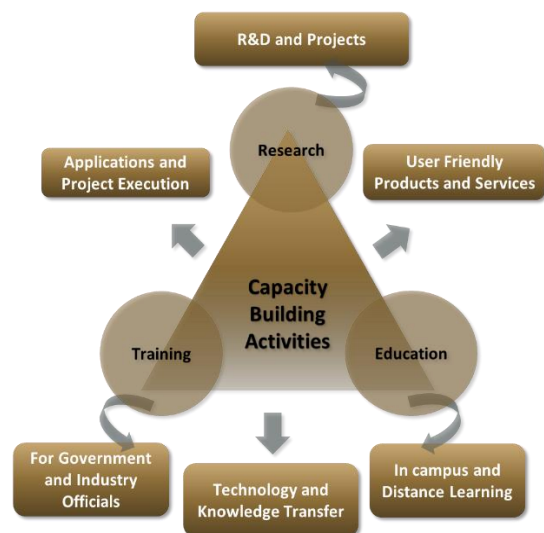
The geo-spatial solutions are packaged and provided as software products for ease of retrieval of spatial and feature attributes data, enabling wide range of analysis, prediction and decision support services. Software technologies like Java, Python, JavaScript, Spring Boot, React JS, Node JS, Microservices, XML, GeoJSON, GeoServer, QGIS, Android and iOS frameworks are used to develop user-friendly and reliable solutions.



Academy of Geo-informatics

BISAG-N, being a national institute for geo-spatial and emerging technology-based applications, has a strong focus on research, development, capacity building and employability support for under-graduate, post-graduate and doctoral students. The institute's infrastructure and supportive work environment provide an ideal platform for students pursuing thesis work in a variety of technologies and domains.

BISAG-N is equipped with cutting-edge infrastructure to support research across multiple disciplines. Dedicated laboratories for research on the application of artificial intelligence, machine learning, GIS (Geographic Information Systems), remote sensing, quantum computing, IoT (Internet of Things) and other emerging technologies are established high-performance computing facilities for intensive data processing tasks are available in the institute. Open-source platforms and in-house developed software are available for projects, research and innovation activities. More than 300 students are supported every year for learning practical based aspects of geo-spatial and emerging technology applications while completing their projects.



BISAG-N Finishing School Programme (FSP) supports wards of special groups of society such as scheduled tribe, scheduled caste, non-gazetted security, armed forces citizens and martyr etc, for development of technical and soft skills to improve their employability including support for entrepreneurship and employment opportunities.

The school provides basic and specialized training to help participants develop capacities as required by the for related industry/domain. The participants undergo thorough counselling sessions as well as mock/actual interviews for improving employability in relevant industries as per the interest of the candidates. Along with the skills training the participants are also allowed to observe engaged in ongoing projects of organization. BISAG-N academic activities also provide capacity building support to government officials for effectively using geo-spatial solutions and decision support systems for performing planning and monitoring for various government schemes and initiatives.

Industrial visits of school students are organized to provide them exposure of space applications and geo-informatics thereby supporting the development of scientific temperament at early stage. More than 5000 students visit the premises every year to get exposure of the activities being carried out at BISAG-N.



Standardization Directorate

BISAG-N has a dedicated directorate for ensuring continuously focus on innovation, standardization and institutionalization of solutions & service. The Standardization Directorate focuses on implementation of various national and international standards related to program solutions and data management. The directorate develops internal SOPs and policies for compliance to these standards and provides training for effective implementation. The directorate also liaison with the various departments and external agencies during internal and external audits. In addition, the directorate acts as member in standards committees of Bureau of Indian Standards (BIS) and other standard developing bodies for development of standards.

Goals of Standardization Directorate include:

- To proactively implement product and process improvements initiatives using various national & international standards and best practices. This includes ensuring quality and reliability of geo-spatial data, completeness, updation of metadata, functional performance and security reviews of applications, capacity building support to end-users for adoption of solutions and information security-oriented culture development
- To develop a culture of efficiently working as per standards while fostering innovation
- To perform reviews, gap analysis, training, documentation and standards implementation support activities

Administration Directorate

Administrative Directorate provides strategic and management support to various activities of BISAG-N. The directorate ensures infrastructure and facilities management in addition to providing services such as coordination during visits of various delegations, logistic support to BISAG-N staff during travel, accounting and finance, procurement, training, safety and security, legal compliance, maintenance, housekeeping, horticulture, construction and extension projects, campus monitoring, canteen management as well as celebrations and welfare activities. The directorate also ensure recruitment and human resource development as per the needs of various projects and directorates while upholding principles of equality, fairness and diversity in work force.

BISAG-N has adequate human resources, well trained for behavioral, functional and domain competencies, with clear roles and responsibilities in alignment with service delivery expectations. The staff well-being and motivation initiatives are ensured for maintaining a culture of shouldering responsibilities proactively and delivering high-performance utilizing multi-disciplinary knowledge.

Administration Directorate ensure strategic and compliance support to operations of the institute by managing human resources, facilities, expansion projects, finance and governance aspects. The directorate also coordinates with Council stakeholders and members of Governing and Executive committee as well as other high-level committees.

Defence Applications Directorate

This directorate develops space, geo-spatial, AI and other emerging technologies-based applications under the guidance of various agencies of the Ministry of Defence, Government of India. All the technologies used are developed inhouse of customized open source. Considering the strategic significance of these projects, dedicated team and premises are allocated for ensuring information security. Most of the critical activities and data management are performed directly at various defence establishments.

Besides the projects of various infrastructure and social welfare ministries and departments of central and state government, BISAG-N has a dedicated directorate for supporting defence and security departments for providing data products, decision support systems and services involving geo-spatial and various other emerging technologies. The projects are executed in high security dedicated laboratory for various units of army, navy and air-force for supporting strategic and operational aspects of the operations.

Through its innovative geospatial solutions and technological expertise, BISAG-N is playing a crucial role in enhancing the nation's defence capabilities and ensuring long-term strategic autonomy. Due to nature of high secrecy of these strategic projects, they are not included in details in this compilation.

Research and Development

BISAG-N's research and development group supports all other directorates for providing solutions related to new definitions and technology. The group develops innovative applications, features, tools and services utilizing technologies such as remote sensing, photogrammetry, artificial intelligence, machine learning, internet of things, computer vision, big data and quantum computing, which are later integrated in live projects.

BISAG-N has made significant progress for the development of Artificial Intelligence/ Machine Learning (AI/ML) applications. Algorithms and models have been developed which can be customized and used with training, validation and live data upon their data for accurate predictions and smarter results.

The applications developed by the R&D team include support for the pluggable APIs which can use the machine learning with minimal code reconfiguration. Various GIS applications are developed at BISAG-N. Utilizing spatial and non-spatial processing of Big Data and Web GIS architecture These applications support multi-terabyte datasets and large number of consumers. Big Data applications for visualization developed by the institute has enabled several projects to explore and analyse multiple large archives. Through continuous research and development, BISAG-N's R&D capabilities are continuously enhanced. This allows to enhance users' satisfaction, growth and success stories.

Cyber Security Cell

Cyber Cell reviews and update the threat landscape for the BISAG-N's critical infrastructure on a regular basis in line with various vulnerabilities detected and published by reputed repositories. The cell verifies the applications using Vulnerability Assessment & Penetration Testing (VAPT) ensuring the compliance to Guidelines for Indian Government Websites (GIGW), Open Web Application Security Project (OWASP), SANS 25 etc. The cell also ensures implementation of good information security practices in line with ISO 27001 such as serious logs review, vulnerability analysis, exception reporting, maintenance of firewall and other network monitoring tools.

CANDIDATE’S DECLARATION

We declare that the 6th semester summer internship project report entitled “**VEDA (Vehicle Evaluation Diagnostics Agent): Enables proactive Vehicles Maintenance by Predictive Failures and Optimizing Fleet Readiness using AI-Driven Analytics**” is our own work conducted under the supervision of the external guide **Mr. Viraj Choksi** from BISAG-N ([Bhaskaracharya National Institute for Space Applications & Geo-informatics](#)). We further declare that to the best of our knowledge, the report for this project does not contain any part of the work which has been submitted previously for such project either in this or any other institutions without proper citation.

Candidate 1’s Signature

Adhish Prasad

Student ID: 26BISAG0542

Candidate 2’s Signature

Harsh Barbhaya

Student ID: 26BISAG0543

Candidate 3’s Signature

Yash Verma

Student ID: 26BISAG0544

Submitted To:

Institute of Advanced Research-Gandhinagar

ACKNOWLEDGMENT

We are grateful to **Shri T.P. Singh, Principal Advisor & Executive Vice Chairman, Governing Council** and **Dr. Vinay Thakur, Director General (BISAG-N)**, for giving us this opportunity to work the guidance of renowned people of the field of GIS-Based Portal, also providing us with the required resources in the company.

We would like to express our endless thanks to our external guide, **Mr. Viraj Choksi**, and to the Training Cell, **Mr. Punit Lalwani & Mr. Sidhdharth Patel** at Bhaskaracharya National Institute of Space Application and Geo-informatics, for their sincere and dedicated guidance throughout the project development.

Also, our hearty gratitude to our Head of Department, **Dr. Sanjay Sonar** and our internal guide **Dr. Zalak Thakker** for giving us encouragement and technical support on the project.

Adhish Prasad.

Student ID: 26BISAG0542

Harsh Barbhaya.

Student ID: 26BISAG0543

Yash Verma.

Student ID: 26BISAG0544

ABSTRACT

VEDA (Vehicle Evaluation Diagnostics Agent): Enables Proactive Vehicle Maintenance by Predicting Failures and Optimizing Fleet Readiness Using AI-Driven Analytics is an advanced Agentic AI and predictive maintenance platform developed to improve the availability, mission readiness, and operational efficiency of military ground vehicle fleets. Modern military vehicles continuously generate large volumes of sensor data, but analyzing this data to identify abnormal conditions and predict **Remaining Useful Life (RUL)** can be time consuming and heavily dependent on manual intervention. This can lead to delays in identifying potential failures and making maintenance decisions. VEDA addresses this challenge by combining machine learning with an intelligent multi agent system to automate vehicle health assessment and maintenance support.

The platform uses two separate and strictly isolated machine learning pipelines: one designed for **Tank variants** and another for **Logistic/Officer vehicles**. Vehicle telemetry first undergoes preprocessing and feature engineering **before being passed to a Random Forest model**, which acts as the primary anomaly detection and routing mechanism. **The resulting information is then used by LSTM with 1D-CNN and XGBoost models** to estimate the vehicle's Remaining Useful Life. Their prognostic outputs are combined through a vehicle-specific fusion process to provide a final RUL estimate.

Beyond machine learning predictions, VEDA uses six specialized AI agents: Monitoring, Diagnostics, Prognostics, Maintenance Planning, Spare Parts, and Fleet Readiness. These agents work together through a FastAPI based backend to interpret vehicle health information and support maintenance related decisions. The results are presented through a React based Command Center, allowing users to follow the vehicle assessment process from data upload and preprocessing through anomaly classification, RUL prediction, fusion, and final readiness assessment.

Overall, VEDA provides a structured and scalable approach to military vehicle health monitoring by combining predictive machine learning with agent-based decision support. The system is designed to reduce dependence on manual analysis, provide timely information about potential vehicle degradation, and support data driven maintenance and fleet readiness decisions.

CONTENTS

Chapter No.	Description	Page No.
-	List of Figures	I
-	List of Tables	II
-	List of Abbreviations	III
-	List of Definitions	IV
-	List of Screenshots	V
1	Introduction	1
1.1	Project Details	1
1.2	Purpose	1
1.3	Scope	2
1.4	Objectives	3
1.5	Technology Stack Used	4
1.6	Literature Review	4
2	Project Management	6
2.1	Feasibility Study	6
2.1.1	Technical Feasibility	7
2.1.2	Time Schedule	7
2.1.3	Operational Feasibility	7
2.1.4	Implementation Feasibility	7
2.2	Project Planning	8
2.2.1	Development Approach and Justification	8
2.2.2	Milestones and Deliverables	8
2.2.3	Roles and Responsibilities	9
2.2.4	Group Dependencies	9
2.3	Project Scheduling (Gantt/PERT Chart)	11
3	System Requirement Study	12
3.1	Existing System Overview	12
3.2	Limitations of Existing System	13
3.3	User Characteristics	13
3.4	Functional Requirements	14
3.5	Non-Functional Requirements	15
3.6	Hardware and Software Requirements	16
3.7	Constraints	16
3.7.1	UI Constraints	17
3.7.2	Communication Interface	17
3.7.3	Hardware interface	17
3.7.4	Criticality of Application	17
3.7.5	Safety and Security Considerations	17
3.8	Assumption and Dependencies	18
4	Proposed System Requirements	19
4.1	Overview of Proposed System	19
4.2	Module Description	19
4.2.1	Module A: TANK Machine Learning Pipeline	19
4.2.2	Module B; Logistic/Officers Machine Learning Pipeline	19
4.2.3	Module C: Agentic AI Orchestration Layer	19

4.3	System Features	20
4.4	Advantage of Proposed System	21
5	System Design	22
5.1	System Architecture Design	22
5.2	UML Diagrams	22
5.2.1	ER diagram	23
5.2.2	Use Case Diagram	24
5.2.3	Class Diagram	24
5.2.4	Sequence Diagram	25
5.2.5	Activity Diagram	26
5.2.6	DFD Diagram	28
5.2.7	Deployment Diagram	29
5.3	Database Design	30
5.3.1	Table Design and Relationship	30
5.3.2	Normalization	30
5.3.3	Data Dictionary	31
5.4	GUI Design	32
5.5	Screenshots	33
6	Implementation Planning	35
6.1	Implementation Environment	35
6.2	Tools and Technology Used	36
6.3	Coding Standard Followed	37
6.3.1	Project and Folder Structure	38
6.3.2	Naming and Formatting Conventions	38
6.3.3	Error Handling and API Responses	39
6.3.4	Documentation and Comments	39
7	Limitations and Future Scope	40
7.1	Limitations	40
7.2	Future Scope	40
8	Conclusion and References	42
8.1	Conclusion	42
8.2	References	42
9	Appendices	43
10	Report Verification Procedure	-

List of Figures

Figure No.	Figure Description	Page No.
Figure 1.3	VEDA Scope Overview	3
Figure 2.2.4	Module Dependency Diagram	3
Figure 2.3	Gantt Chart of Project Schedule	15
Figure 3.1	Existing (Manual) Method vs Proposed (Automated) Method	16
Figure 4.3	System Features Map	17
Figure 5.2.1	ER Diagram	29
Figure 5.2.2	Use Case Diagram	31
Figure 5.2.3	Class Diagram	32
Figure 5.2.4	Sequence Diagram	33
Figure 5.2.5	Activity Diagram	34
Figure 5.2.6	Data Flow Diagram- Level 0 and Level 1	34
Figure 5.2.7	Deployment Diagram	35
Figure 6.1	Local Development Setup/ Service Ports Diagram	36
Figure 8.2	Future Scope Roadmap (Indicative Priority Order)	47

List of Tables

Table No.	Table Description	Page No.
Table 1.1	Project Details (Title, Organization, Role, Duration, Team Size, Mentor)	1
Table 1.5	Technology Stack Used in the Proposed System	4
Table 1.6	Comparison with Existing Approach	5
Table 2.1	Feasibility Study Summary	6
Table 2.2.2	Project Milestones and Deliverables	8
Table 2.2.3	Roles and Responsibilities	9
Table 2.2.4	Module Dependency Matrix	10
Table 3.2	Limitations of the Existing System	13
Table 3.3	User Characteristics	14
Table 3.4	Functional Requirements of the Proposed System	14
Table 3.5	Non-Functional Requirements of the Proposed System	15
Table 3.6	Hardware & Software Requirements	16
Table 3.7	Constraints Summary	16
Table 3.8	Assumptions & Dependencies	18
Table 4.2	Modules & Functions of the Proposed System	20
Table 4.4	Advantages of the Proposed System	21
Table 5.2	UML Diagram Types & Purpose	23
Table 5.3.1	PostgreSQL Collections & Relationships	30
Table 5.3.2	PostgreSQL and TimescaleDB Schema Design Decisions	31
Table 5.3.3.1	Data Dictionary (vehicles)	32
Table 5.3.3.2	Data Dictionary (telemetry_logs)	32
Table 5.3.3.3	Data Dictionary (ml_infernces)	32
Table 5.3.3.4	Data Dictionary (agent_events)	33
Table 6.1	Implementation Environment	36
Table 6.2	Tools and Technology Used	38
Table 6.3.1	Project and Folder Structure	39
Table 6.3.2	Naming and Formatting Conventions	40
Table 8.1	Limitations Summary	46
Table 8.2	Future Scope Summary	47
Table B1	Appendix B: Environment Variable Reference	50
Table C1	Appendix C: Supported Vehicles Models and Pipelines	50
Table D1	Full Agent Action List	51

List of Abbreviations

Abbreviation	Full Form
VEDA	Vehicle Evaluation Diagnostic Agents
1D CNN	One-Dimensional Convolutional Neural Network
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
ER	Entity-Relationship
GUI	Graphical User Interface
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
ORM	Object-Relational Mapping
REST	Representational State Transfer
RF	Random Forest
RMSE	Root Mean Square Error
RUL	Remaining Useful Life
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
XGBoost	extreme Gradient Boosting

List of Definitions

Term	Definition
Agentic AI	A system design in which specialized AI agents work with machine learning outputs to support analysis, decision-making, and operational tasks with limited continuous human intervention.
Data Leakage	The seven SDLC methodologies the platform supports, each with its own terminology, structure, and required document fields.
Diagnostics	The process of analyzing vehicle telemetry and anomaly-related outputs to identify abnormal conditions, possible faults, and their underlying causes.
Dual Pipeline Strategy	VEDA's approach of maintaining two independent machine learning pipelines: one for Tank vehicles and another for Logistic/Officer vehicles, with separate datasets, feature contracts, preprocessing, and model Fleet Readiness Policy
Fleet Readiness Policy	A defined set of rules used by the Fleet Readiness Agent to convert vehicle health and RUL information into operational readiness states such as READY, ATTENTION, NOT_READY, or UNKNOWN.
Frozen Artifact	A trained machine learning model, preprocessing object, or configuration file that has completed the required validation and is locked to preserve consistent and reproducible inference behavior.
Fusion	The process of combining RUL predictions from multiple prognostic models using predefined and validation-selected weights to obtain a final RUL estimate.
Prognostics	The process of using historical and current time-series information to estimate future vehicle health and Remaining Useful Life (RUL).
Remaining Useful Life (RUL)	The estimated remaining operational time of a vehicle based on its observed telemetry and predicted degradation condition.
Rolling Window	A time-series preprocessing technique in which statistical information is calculated over a fixed number of consecutive observations and the window moves forward through the dataset to capture temporal behavior.
Telemetry	Time-stamped operational data collected from vehicle sensors, such as engine speed, temperature, pressure, vibration, fuel level, and other vehicle-condition measurements.

List of Screenshots

Screenshot No.	Screenshot Description	Page No.
Screenshot 5.4	VEDA GUI: Dual-Pipeline Vehicle Selection Dashboard	33
Screenshot 5.5 (i)	VEDA landing page	34
Screenshot 5.5 (ii)	VEDA: TANK Command Center	34
Screenshot 5.5 (iii)	VEDA: TANK telemetry data analysis result	35
Screenshot 5.5(iv)	VEDA: Vehicle inference history	35

1 Introduction

This project focuses on the challenge of monitoring the health of military ground vehicles, identifying abnormal conditions, and predicting their Remaining Useful Life (RUL) in a timely and reliable manner. VEDA addresses this challenge by combining machine learning-based diagnostics with an Agentic AI framework that helps convert vehicle telemetry into meaningful health and fleet readiness information. The following sections describe the project's purpose, scope, objectives, system architecture, machine learning models, agentic AI framework, technology stack, implementation, and performance evaluation.

1.1 Project Details

Attribute	Detail
Project Title	VEDA (Vehicle Evaluation Diagnostic Agent)
Organization	BISAG-N (Bhaskaracharya National Institute for Space Applications and Geo-informatics), Gandhinagar
Role	AI/ML Intern
Duration	17 th June 2026 to 16 th September 2026
Team Size	3
Mentor / Guide	Viraj Choksi

Table 1.1: Project Details (Title, Organization, Role, Duration, Team Size, Mentor)

1.2 Purpose

The purpose of VEDA is to make military vehicle health monitoring more intelligent, automated, and reliable. The system is designed to analyze vehicle data, detect abnormal conditions, predict the Remaining Useful Life (RUL), and provide useful information for maintenance and fleet readiness decisions. By reducing the need for continuous manual analysis, VEDA aims to help identify potential vehicle issues earlier and support better decisions for maintaining vehicle availability and operational readiness.

1.3 Scope

VEDA covers the complete process of analyzing vehicle data, applying machine learning models, and using the resulting predictions to support vehicle health and maintenance assessment. The major areas covered by the project are:

1.3.1 Telemetry Processing:

Collection and preprocessing of multivariate time series vehicle data, including parameters such as temperature, pressure, vibration, engine performance, and other operational measurements.

1.3.2 Dual Pipeline Machine Learning:

Development of two independent machine learning pipelines, one for **Tank vehicles** and another for **Logistic/Officer vehicles**. The pipelines use **Random Forest** for normal and abnormal condition classification and routing, followed by **LSTM + 1D CNN** and **XGBoost** models for Remaining Useful Life (RUL) prediction. The prognostic outputs are then combined using the respective fusion configurations.

1.3.3 Agentic AI Orchestration:

Integration of six specialized agents **Monitoring, Diagnostics, Prognostics, Maintenance Planning, Spare Parts, and Fleet Readiness**. These agents process the available model outputs and vehicle information to generate structured assessments and support maintenance and fleet-readiness decisions according to the defined policies, including the demo_v1 readiness policy.

1.3.4 Command Center Interface:

Development of a full-stack web application using a **FastAPI backend and React frontend** to allow users to upload vehicle data, follow the processing pipeline, and view vehicle health, RUL predictions, and agent-generated assessments through an interactive Command Center.

The current version of VEDA is focused on validating the system architecture and predictive maintenance workflow using **synthetic telemetry datasets**.

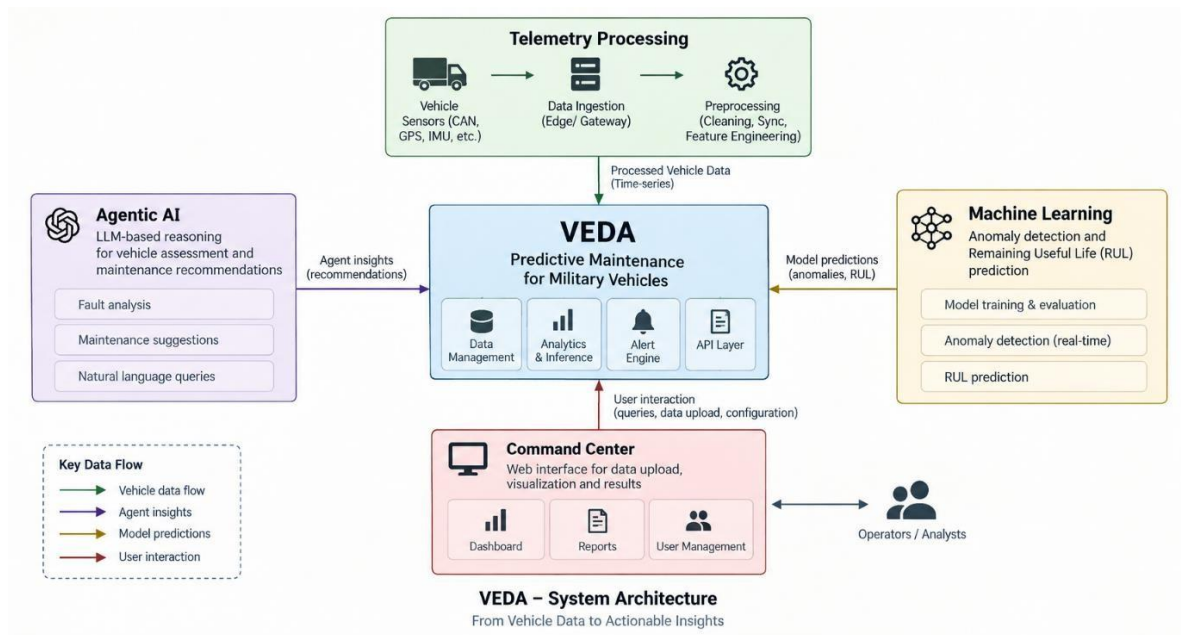


Figure 1.1: VEDA Scope Overview

1.4 Objectives

- 1.4.1 To develop an automated system for monitoring military ground vehicles and analyzing their telemetry to assess vehicle health.
- 1.4.2 To detect abnormal conditions and predict Remaining Useful Life (RUL) using separate machine learning pipelines for Tank and Logistic/Officer vehicles.
- 1.4.3 To combine machine learning with Agentic AI through six specialized agents for diagnostics, prognostics, maintenance planning, spare parts, monitoring, and fleet readiness.
- 1.4.4 To integrate the complete system through a FastAPI backend and React based Command Center for data processing and result visualization.
- 1.4.5 To evaluate the reliability of the VEDA platform through model validation, leakage checks, artifact verification, integration testing, and system level testing.

1.5 Technology Stack Used

The VEDA system is designed as a modular pipeline, with separate components for vehicle data processing, machine learning, and Agentic AI-based decision support. This structure keeps the different parts of the system organized and allows them to work together efficiently.

Layer	Technology
Frontend	React (TypeScript), Motion.dev (for animations)
Application services / API	Python, FastAPI
Machine Learning (Inference)	scikit-learn (Random Forest), TensorFlow/Keras (LSTM + 1D-CNN), XGBoost
Agentic AI Orchestration	Python (Custom Six-Agent Orchestration Layer)
Database	PostgreSQL + TimescaleDB, SQLAlchemy (ORM)
Validation & Integrity	Vehicle-Level Leakage Audits, Frozen Artifact Reload Verification

Table 1.2: Technology Stack Used in the Proposed System

The frontend uses React and TypeScript for the Landing Page and Command Center, while the FastAPI backend manages data processing and routes telemetry through the appropriate Tank or Logistic/Officer machine learning pipeline. The resulting RUL predictions are then passed to the six-agent orchestration layer for further assessment, with PostgreSQL and TimescaleDB used to store telemetry, inference records, and agent-related data.

1.6 Literature Review.

Predictive maintenance has increasingly moved from scheduled and manual inspection toward data-driven vehicle health monitoring. Machine learning models such as Random Forest, LSTM, 1D-CNN, and XGBoost can analyze vehicle telemetry, detect abnormal conditions, and estimate Remaining Useful Life (RUL). Agentic AI can further help interpret these predictions and support maintenance and fleet-readiness decisions. VEDA combines these approaches into an integrated platform with separate machine learning pipelines for Tank and Logistic/Officer vehicles, followed by six specialized agents for monitoring, diagnostics, prognostics, maintenance planning, spare-parts information, and fleet readiness.

Area	Existing Approach	VEDA Approach
Vehicle Monitoring	Periodic inspections and scheduled maintenance	Analysis of available vehicle telemetry
Anomaly Detection	Manual or conventional fault detection	Random Forest-based anomaly classification and routing
RUL Prediction	Individual predictive models	LSTM + 1D-CNN and XGBoost-based prediction
Model Combination	Often based on a single model	Vehicle-specific fusion of prognostic outputs
Vehicle Categories	Generalized models across vehicle types	Separate pipelines for Tank and Logistic/Officer vehicles
Maintenance Support	Manual interpretation of results	Six specialized Agentic AI agents
Fleet Readiness	Manual fleet assessment	Fleet Readiness Agent with defined readiness policies
System Integration	Separate tools and model implementations	Integrated FastAPI backend and React-based Command Center
Data Processing	Manual or semi-automated processing	Automated preprocessing and inference workflow
Uncertain Information	Manual interpretation may be required	Explicit handling using UNKNOWN and NOT_AVAILABLE states

Table 1.3: Comparison with Existing Approach

2 Project Management

The VEDA project followed an iterative Agile approach, allowing the machine learning pipelines, backend, and frontend to be developed in parallel. Verification steps such as vehicle-level leakage checks and frozen model reload tests were included throughout the development process to maintain data integrity and ensure smooth integration of the system components.

2.1 Feasibility Study

Feasibility Type	Verdict
Technical (2.1.1)	Feasible built using established technologies such as TensorFlow, XGBoost, scikit-learn, and FastAPI for integrating the ML and Agentic AI components.
Time Schedule (2.1.2)	Feasible within the internship duration the project was divided into well-defined phases covering data preparation, ML development, backend integration, and frontend development.
Operational (2.1.3)	Feasible simplifies vehicle health analysis by converting RUL and diagnostic results into clear statuses such as READY, ATTENTION, and NOT_READY.
Implementation (2.1.4)	Feasible defined API contracts, Pydantic schemas, and verification checks supported reliable integration of the ML models with the Agentic AI layer.

Table 2.1: Feasibility Study Summary

2.1.1 Technical Feasibility

VEDA is technically feasible because its architecture is based on well-established and widely used technologies such as TensorFlow, XGBoost, and scikit-learn. These frameworks provide the required capabilities for developing the vehicle-specific machine learning pipelines, including abnormal-condition classification and Remaining Useful Life (RUL) prediction.

The system also uses a Python FastAPI backend to connect the machine learning models with the Agentic AI layer. This modular approach keeps the different components separated while allowing them to communicate through defined interfaces, making the overall system easier to integrate, test, and maintain.

2.1.2 Time Schedule Feasibility

The VEDA project was divided into clear development phases, beginning with data preparation and preprocessing, followed by the independent development of the Tank and Logistic/Officer machine learning pipelines. This phased approach helped keep the development organized and allowed different components to be worked on in parallel.

The latter stages focused on backend integration, Agentic AI orchestration, and frontend development. By following this structured sequence, the major development and testing activities could be completed within the available internship timeline.

2.1.3 Operational Feasibility

VEDA is operationally feasible because it converts complex vehicle health and RUL predictions into simple and understandable readiness statuses such as READY, ATTENTION, and NOT_READY. This makes the system output easier to interpret without requiring the user to understand the underlying machine learning calculations.

The Command Center interface presents the results in a clear and visual manner, helping users quickly understand the vehicle's health and readiness condition. This supports faster interpretation and reduces the need for continuous manual analysis.

2.1.4 Implementation Feasibility

VEDA is feasible to implement because the system uses defined data schemas, API contracts, and structured interfaces between its different components. Pydantic schemas and the FastAPI backend help maintain consistent communication between the machine learning and Agentic AI layers.

The development process also included vehicle-level leakage checks, model validation, and frozen artifact verification to ensure reliable and consistent results. These checks helped make the integration structured, reproducible, and suitable for the intended vehicle health monitoring.

2.2 Project Planning

2.2.1 Development Approach & Justification

The VEDA project followed an iterative Agile methodology, which allowed the system to be developed and improved in manageable stages. This approach was particularly useful for the machine learning components, where continuous training, evaluation, and validation were required before the final model artifacts could be frozen and integrated into the backend.

The development process also allowed issues identified during validation to be addressed before moving to the next stage. For example, the initial Logistic/Officer pipeline was reviewed and improved to develop the V2 pipeline before its final artifacts were frozen.

2.2.2 Milestones & Deliverables

The VEDA project was developed through five major milestones, covering data preparation, vehicle-specific machine learning pipelines, Agentic AI integration, backend development, and frontend implementation. Each milestone produced validated components that were used as the foundation for the next stage of development.

Milestone	Deliverable
Telemetry Data Processing and Feature Engineering	Standardized Tank and Logistic/Officer telemetry datasets with engineered rolling window and delta features and fitted preprocessing components.
Tank ML Pipeline Freeze	Validated and serialized Tank Random Forest, LSTM + 1D CNN, and XGBoost models with finalized 0.30/0.70 fusion weights.
Logistic/Officer V2 ML Pipeline Freeze	Corrected and validated V2 Random Forest, LSTM + 1D CNN, and XGBoost models with finalized 0.30/0.70 fusion weights.
Backend API & Agentic Orchestration	FastAPI application for telemetry routing, ML inference, RUL fusion, and coordination of the six specialized AI agents.
Command Center Frontend Integration	React and Motion.dev interface with interactive vehicle visualization, telemetry upload, and backend API integration.

Table 2.2.2: Project Milestones and Deliverables

2.2.3 Roles & Responsibilities

The work on the VEDA project was divided among three team members, with each person responsible for a major part of the system. This helped the team work in parallel while keeping the integration between the AI, backend, database, and frontend components organized.

Role (Team Member)	Responsibilities
AI and Agentic AI (Yash Verma)	Worked on the machine learning models for Tank and Logistic/Officer vehicles, including model development, validation, leakage checks, and finalization. Also developed the AI agents, Agent framework, agent pipeline, and overall Agent orchestration.
Backend and Database (Adhish Prasad)	Handled the FastAPI backend integration , connecting the ML and Agentic AI components with the application. Also worked on the PostgreSQL/TimescaleDB database , API integration, and backend services.
Frontend (Harsh Barbhaya)	Developed the React and TypeScript frontend , including the Command Center, telemetry upload interface, vehicle health visualization, Motion.dev-based vehicle presentation, and connection with the backend APIs.

Tables for roles and Responsibilities.

2.2.4 Group Dependencies

The VEDA project required close coordination between the three team members, as each part of the system depended on the output of the previous stage. The main dependencies were between the **AI, Backend, and Frontend** components.

Dependency	Description
Backend on AI/ML	The Backend depended on the finalized and validated ML models, preprocessing components, and fusion weights before integrating them into the inference service.
Agentic AI on ML Outputs	The Agentic AI layer depended on standardized outputs such as RF abnormality results and fused RUL values from the ML pipeline. Changes to these outputs or feature contracts required corresponding updates to the agent pipeline.
Frontend on Backend	The Frontend depended on the finalized REST API endpoints and response structures from the Backend to integrate the Command Center and display the processed vehicle intelligence.

Table 2.2.4: Module Dependency Matrix

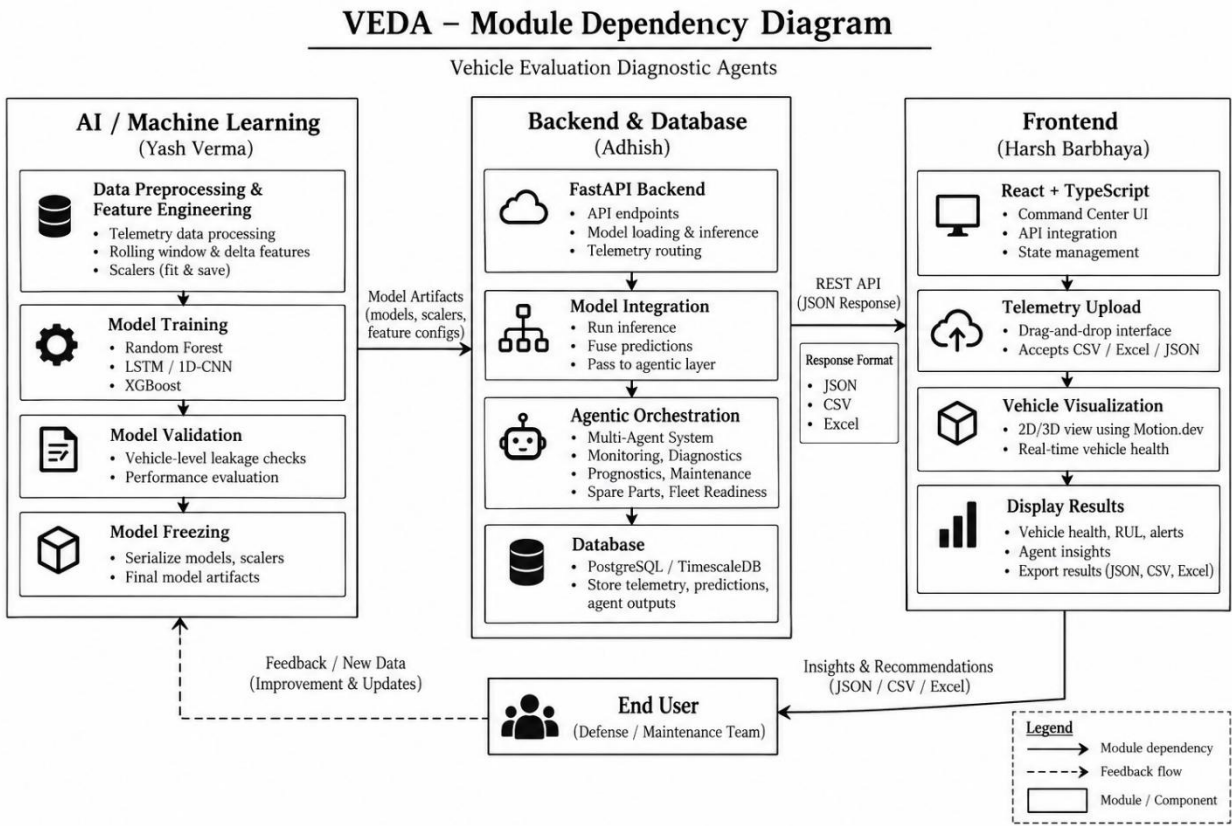


Figure 2.2.4: Module Dependency Diagram

2.3 Project Scheduling (Gantt/PERT Chart)

The Gantt Chart shows the planned development schedule for the VEDA predictive maintenance and vehicle health monitoring system over the twelve-week internship period. It highlights the major activities carried out during the project, including data preparation, machine learning development, backend and Agentic AI integration, frontend development, testing, and final documentation.



Figure 2.3: Gantt Chart of the Project Schedule

3 System Requirement Study

The requirement study for VEDA focused on understanding the existing approach to military vehicle health monitoring, identifying its key limitations, and defining the needs of its intended users. Based on this study, the system requirements were defined across functional, non-functional, hardware and software, and project constraint areas to guide the overall design and development of VEDA.

3.1 Existing System Overview

At present, the health of military ground vehicles is largely monitored through **manual telemetry analysis and reactive maintenance practices**. Maintenance personnel generally review sensor data after vehicle operation or take action when predefined limits, such as high engine temperature, are reached. This approach provides limited information about problems before they become serious.

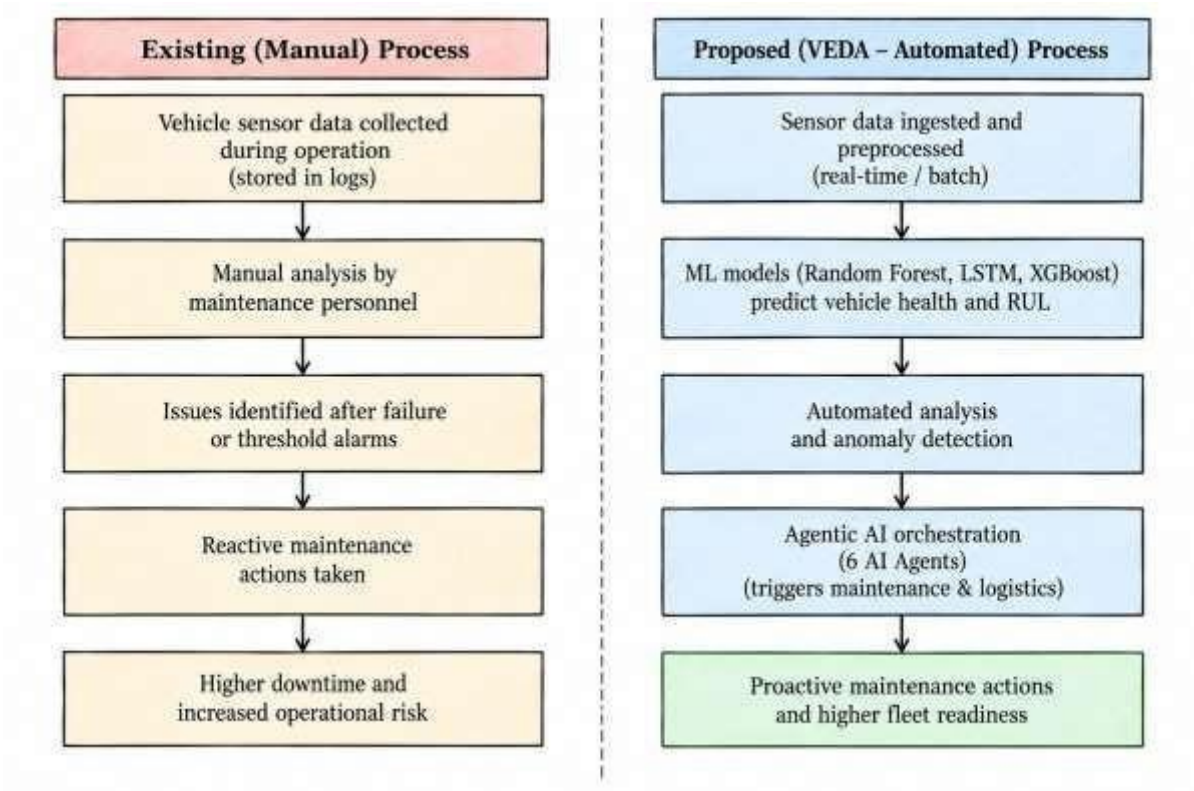


Figure 3.1: Existing (Manual) Process vs. Proposed (Automated) Process

3.2 Limitations of the Existing System

The existing approach has several limitations. Faults are often identified only after they occur, which can result in unexpected vehicle downtime. At the same time, analyzing large amounts of sensor data manually requires considerable expertise and can be affected by human error and fatigue. In addition, vehicle health prediction, maintenance planning, and spare-parts management are generally handled separately, making it difficult to get a combined view of the vehicle's overall condition.

Limitation	Description
High Latency & Reactive Nature	Faults may be identified only after they occur or when predefined limits are exceeded, increasing the risk of unexpected downtime.
Human Bottlenecks	Manual analysis of complex, multi-variable telemetry requires significant expertise and can be affected by fatigue or human error.
Siloed Operations	Vehicle health prediction, maintenance planning, and spare parts activities are handled separately without a common orchestration layer.

Table 3.2: Limitations of the Existing System

3.3 User Characteristics

The primary users of VEDA are expected to include military logistics commanders, fleet operators, and maintenance technicians. These users need quick and easy-to-understand information about vehicle health and readiness to support deployment and maintenance decisions. Therefore, VEDA presents complex ML results through simple outputs such as READY, ATTENTION, and NOT_READY, reducing the need for users to have specialized knowledge of machine learning or data science.

Limitation	Description
Admin	System-oriented user responsible for overall platform management and monitoring.
Fleet Operator	Operational user focused on vehicle condition, fleet status, and readiness information.
Maintainer	Technical user focused on vehicle faults, health information, RUL, and maintenance-related results.

Table 3.3: Limitations of the Existing System

3.4 Functional Requirements

The functional requirements define the main operations that VEDA must perform, from receiving vehicle telemetry to generating maintenance and fleet-readiness information. The system is designed to process Tank and Logistic/Officer vehicles through their respective ML pipelines and then use the resulting predictions within the Agentic AI layer.

Req. ID	Requirement
FR-01	Allow users to upload vehicle telemetry data and route it to the appropriate Tank or Logistic/Officer pipeline.
FR-02	Classify vehicle telemetry as normal or abnormal using the vehicle-specific Random Forest model.
FR-03	Estimate the vehicle's Remaining Useful Life (RUL) using the LSTM + 1D-CNN and XGBoost models.
FR-04	Combine the RUL predictions using the predefined and validated vehicle-specific fusion weights.
FR-05	Process ML results through the six specialized agents for monitoring, diagnostics, prognostics, maintenance planning, spare parts, and fleet readiness.
FR-06	Apply the defined Fleet Readiness policy to assign READY , ATTENTION , NOT_READY , or UNKNOWN status.
FR-07	Return UNKNOWN/NOT_AVAILABLE for spare-parts information when reliable inventory data is unavailable.
FR-08	Display vehicle health, RUL, diagnostic results, maintenance information, and fleet-readiness results through the Command Center.

Table 3.4: Functional Requirements of the Proposed System

3.5 Non-Functional Requirements

Category	Requirement
Isolation & Integrity	The system must maintain complete isolation between the Tank and Logistic/Officer ML pipelines. Feature contracts, model weights, preprocessing components, and fusion configurations must not be cross-contaminated during execution.
Determinism & No Hallucination	The Agentic AI orchestration layer must operate deterministically. Agents must not invent numerical thresholds, predict inventory. When authoritative information is unavailable, the system must return UNKNOWN or NOT_AVAILABLE .
Performance & Low Latency	The FastAPI backend must process telemetry, perform LSTM/XGBoost inference, execute prognostic fusion, and complete the six-agent orchestration with sufficiently low latency to support near real-time dashboard updates.
Scalability	The PostgreSQL/TimescaleDB-based architecture must be capable of handling large volumes of continuous time-series telemetry generated by multiple military vehicles.
Usability	The React-based Command Center must provide an accessible and intuitive user experience. It should hide complex ML calculations and present actionable vehicle intelligence through a lightweight 2.5D X-ray visualization without relying on heavy 3D. glb assets.

Table 3.5: Non-Functional Requirements of the Proposed System

3.6 Hardware & Software Requirements

Category	Requirement
Frontend framework	React, TypeScript, and Vite
Backend runtime	Python 3.9+, FastAPI, and Uvicorn
AI service runtime	scikit learn (Random Forest), TensorFlow/Keras (LSTM + 1D-CNN), and XGBoost
Database	PostgreSQL v14+ with TimescaleDB extension
Development hardware	Multi core processor (Intel Core i5/i7 or AMD Ryzen 5/7 equivalent or better), minimum 16 GB RAM recommended, and at least 50 GB available SSD storage
GPU Support	FastAPI for API routing and Pydantic for request/response schema validation
Browser support	Modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari

Table 3.6: Hardware & Software Requirements

3.7 Constraints

Constraint Type	Summary
UI (3.7.1)	Lightweight 2.5D X-ray visualization without heavy. glb assets
Communication Interface (3.7.2)	REST/HTTP JSON APIs with Pydantic schemas
Hardware Interface (3.7.3)	Historical, batch, or simulated telemetry; no live CAN bus integration
Criticality of Application (3.7.4)	Frozen and validated models with deterministic agent logic
Safety & Security (3.7.5)	No fabricated information; unavailable data returns UNKNOWN or NOT_AVAILABLE

Table 3.7: Constraints Summary

3.7.1 UI Constraints

The interface must provide a clear and immersive view of the vehicle condition using a lightweight 2.5D X-ray visualization. The frontend should avoid heavy. glb or complex 3D assets and instead use image-based visualization with Motion.dev to maintain smooth interaction and faster loading.

3.7.2 Communication Interface

The frontend communicates with the backend through RESTful HTTP JSON APIs. The API requests and responses follow predefined Pydantic schemas, ensuring that telemetry and vehicle information are exchanged in a consistent and structured format between the frontend, ML inference service, and Agentic AI orchestration layer.

3.7.3 Hardware Interface

The current version of VEDA does not directly connect to vehicle CAN buses, onboard diagnostic systems, or physical sensors. The system is designed to work with historical, batch, or simulated vehicle telemetry provided through file uploads for processing and analysis.

3.7.4 Criticality of Application

VEDA supports the assessment of military vehicle health and operational readiness, making the correctness of its predictions and decisions important. To reduce the risk of incorrect results, the system uses validated and frozen ML artifacts together with deterministic agent logic rather than allowing agents to generate unsupported decisions.

3.7.5 Safety & Security Considerations

- The Agentic AI agents must not invent, assume, or fabricate information that is not supported by the available data.
- Frozen and validated ML models must be used to ensure consistent and reliable predictions during system operation.
- When authoritative spare-parts inventory information is unavailable, the system must return UNKNOWN or NOT_AVAILABLE instead of generating unverified information.
- Tank and Logistic/Officer data and model artifacts must remain isolated to prevent incorrect cross pipeline results.

3.8 Assumptions & Dependencies

The development of VEDA is based on certain assumptions about the quality and structure of the incoming telemetry data and depends on consistent preprocessing and sequential execution of the system components. These assumptions and dependencies help ensure that the ML models and Agentic AI layer produce reliable and consistent results.

Type	Description
Assumption	Uploaded telemetry data follows the expected feature and input contracts for the respective vehicle pipeline.
Assumption	The training data represents normal vehicle behavior sufficiently for identifying abnormal conditions.
Dependency	Model inference depends on applying the same feature engineering and preprocessing used during model training.
Dependency	Agentic AI execution depends on the sequential flow of outputs from Prognostics to Maintenance Planning and Fleet Readiness.

Table 3.8: Assumptions & Dependencies

4 Proposed System Requirements

4.1 Overview of Proposed System

The proposed VEDA (Vehicle Evaluation and Diagnostics Agent) system is an automated predictive maintenance platform designed to improve vehicle health monitoring and reduce dependence on manual analysis. It processes vehicle telemetry through separate Tank and Logistic/Officer machine learning pipelines, using Random Forest for abnormal condition detection and LSTM + 1D-CNN with XGBoost for RUL prediction.

The predicted results are then passed to the Agentic AI layer, where six specialized agents analyze the vehicle condition and generate structured information related to diagnostics, prognostics, maintenance planning, spare parts (for future), and fleet readiness. The complete process is handled through the backend and presented to users through the React-based Command Center in a clear and intuitive manner.

4.2 Module Descriptions

4.2.1 Module A: Tank Machine Learning Pipeline

This module focuses specifically on Tank vehicle data and uses a defined 43-feature input structure. Random Forest is used to identify abnormal conditions, followed by LSTM + 1D-CNN and XGBoost models to estimate RUL. The final prediction uses the frozen Tank fusion configuration of 0.30L STM + 0.70 XGB.

4.2.2 Module B: Logistic/Officer Machine Learning Pipeline

This module is developed separately for Logistic and Officer vehicles to maintain proper data and model isolation. The corrected V2 pipeline uses a 22-feature Random Forest model for abnormal-condition detection and 28-feature LSTM + 1D-CNN and XGBoost models for RUL prediction. Their outputs are combined using the validated 0.30 LSTM + 0.70 XGB fusion configuration.

4.2.3 Module C: Agentic AI Orchestration Layer

This module connects machine learning results with VEDA's decision-making process. It includes six specialized agents that use defined rules to provide maintenance information and assign statuses such as READY, ATTENTION, NOT_READY, or UNKNOWN.

Module	Purpose	Key Functions
Module A: Tank Machine Learning Pipeline	Evaluate and predict the health of Tank vehicles	43-feature processing; Random Forest abnormal-condition detection; LSTM + 1D-CNN and XGBoost RUL prediction; 0.30 LSTM + 0.70 XGB fusion
Module B: Logistic/Officer Machine Learning Pipeline	CEvaluate and predict the health of Logistic/Officer vehicles	22-feature Random Forest classification; 28-feature LSTM + 1D-CNN and XGBoost RUL prediction; 0.30 LSTM + 0.70 XGB fusion
Module C: Agentic AI Orchestration Layer	Convert ML results into actionable vehicle health and readiness information	Monitoring; Diagnostics; Prognostics; Maintenance Planning; Spare Parts; Fleet Readiness; readiness status assignment

Table 4.2: Modules & Functions of the Proposed System

4.3 System Features

- **Automated Dual Pipeline Routing:** The system identifies the vehicle type and automatically routes its telemetry to the appropriate Tank or Logistic/Officer ML pipeline.
- **Predictive Anomaly Detection:** Random Forest models classify vehicle telemetry as normal or abnormal based on the defined vehicle-specific thresholds.
- **Advanced RUL Fusion:** RUL predictions from LSTM + 1D-CNN and XGBoost are combined using validated, vehicle-specific fusion weights.
- **Deterministic Agentic Orchestration:** Six specialized agents process the ML results and generate structured information for diagnostics, maintenance, spare parts, and fleet readiness.
- **Interactive Dashboard:** The React based Command Center allows users to upload telemetry files and view the processed vehicle health results.
- **Safety Fallbacks:** The system avoids unsupported information and returns UNKNOWN or NOT_AVAILABLE when required data, such as spare-parts inventory, is unavailable.

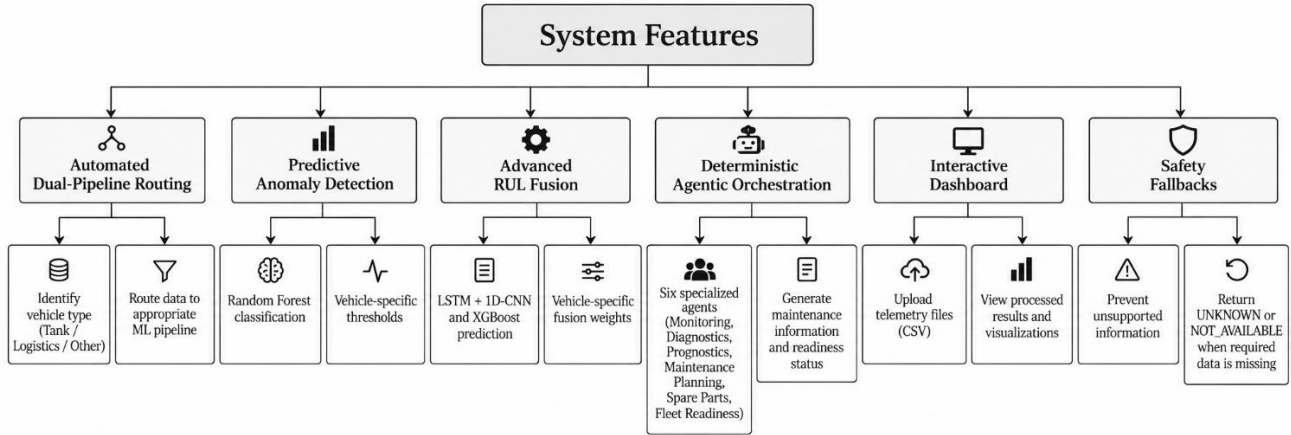


Figure 4.3: System Features Map

4.4 Advantages of Proposed System

VEDA provides several advantages over traditional manual and reactive vehicle monitoring approaches. It combines predictive machine learning with Agentic AI to identify potential issues earlier, maintain separation between vehicle-specific models, and convert complex predictions into simple operational information. The system also follows deterministic safety rules and provides an accessible interface for users.

Advantage	Resolves Limitation
Proactive RUL-based maintenance prediction	Reactive fault detection and unexpected vehicle downtime
Separate vehicle-specific ML pipelines	Risk of mixing Tank and Logistic/Officer models and data
Automated Agentic AI decision support	Manual interpretation of complex ML predictions
Deterministic safety-based decision making	Risk of unsupported or fabricated information
Intuitive Command Center interface	Difficulty in understanding complex telemetry and ML results

Table 4.4: Advantages of the Proposed System

5 System Design

5.1 System Architecture Design

The VEDA system follows a modular architecture with three main layers: **Frontend Presentation**, **API Gateway and ML Inference**, and **Agentic Orchestration and Persistence**. The React-based frontend provides the Command Center for telemetry uploads and visualization, while the FastAPI backend routes data to the appropriate Tank or Logistic/Officer pipeline and runs the ML models to generate RUL predictions. These results are then processed by the six specialized agents through the Veda Orchestrator, while telemetry, model results, and agent events are stored in PostgreSQL with TimescaleDB.

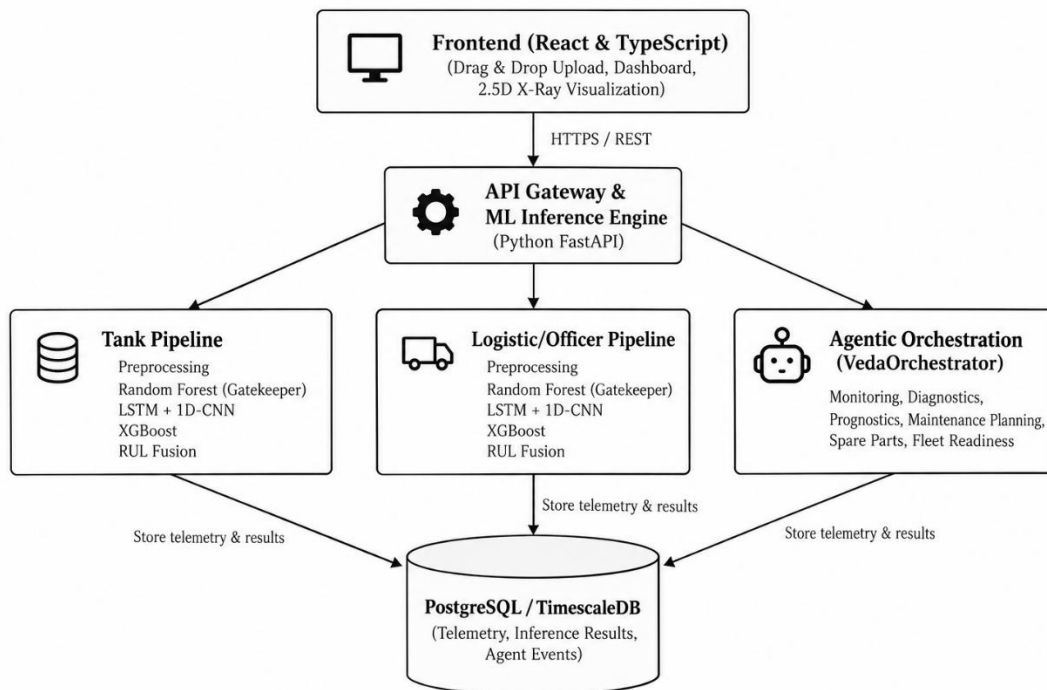


Figure 5.1: System Architecture Diagram

5.2 UML Diagrams

The following diagrams explain VEDA from different viewpoints, showing its **structure**, **user interactions**, **system flow**, **data movement**, and **deployment setup**. Together, they provide a clear understanding of how the different parts of VEDA are organized and work with each other.

Diagram	What It Shows	Section
E-R Diagram	Entity relationships across VEDA database tables	5.2.1
Use Case Diagram	User interactions with the VEDA system	5.2.2
Class Diagram	Structural relationships between core backend and agent classes	5.2.3
Sequence Diagram	Telemetry processing and agent orchestration flow	5.2.4
Activity Diagram	Fleet Readiness decision-making workflow	5.2.5
DFD (Level 0 & 1)	Data flow from telemetry to final system output	5.2.6
Deployment Diagram	Physical and infrastructure placement of system components	5.2.7

Table 5.2: UML Diagram Types & Purpose

5.2.1 E-R Diagram

The E-R model is centered around **Vehicles**, which connects the vehicle details with its telemetry, ML predictions, and agent results. **Telemetry Logs** stores sensor data, **ML Inferences** stores the prediction results, and **Agent Events** stores the information generated by the agents. Primary Keys identify each record, while Foreign Keys connect the related data.

VEDA – Entity Relationship Diagram

Predictive Maintenance for Military Vehicles

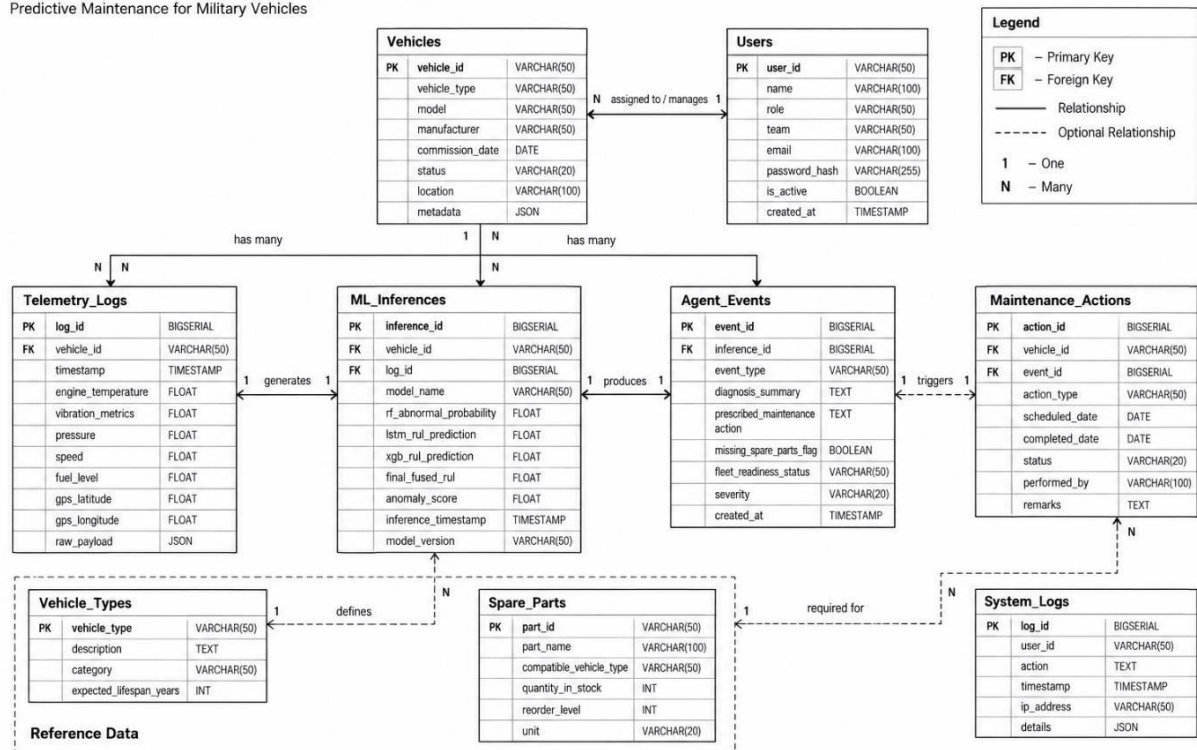


Figure 5.2.1: Entity Relationship Diagram

5.2.2 Use Case Diagram

The VEDA system has three primary user roles, each with a specific focus: the **Admin** manages the overall system and its operation; the **Fleet Operator** monitors vehicle condition, fleet status, and readiness; and the **Maintainer** focuses on vehicle faults, RUL predictions, and maintenance-related information.

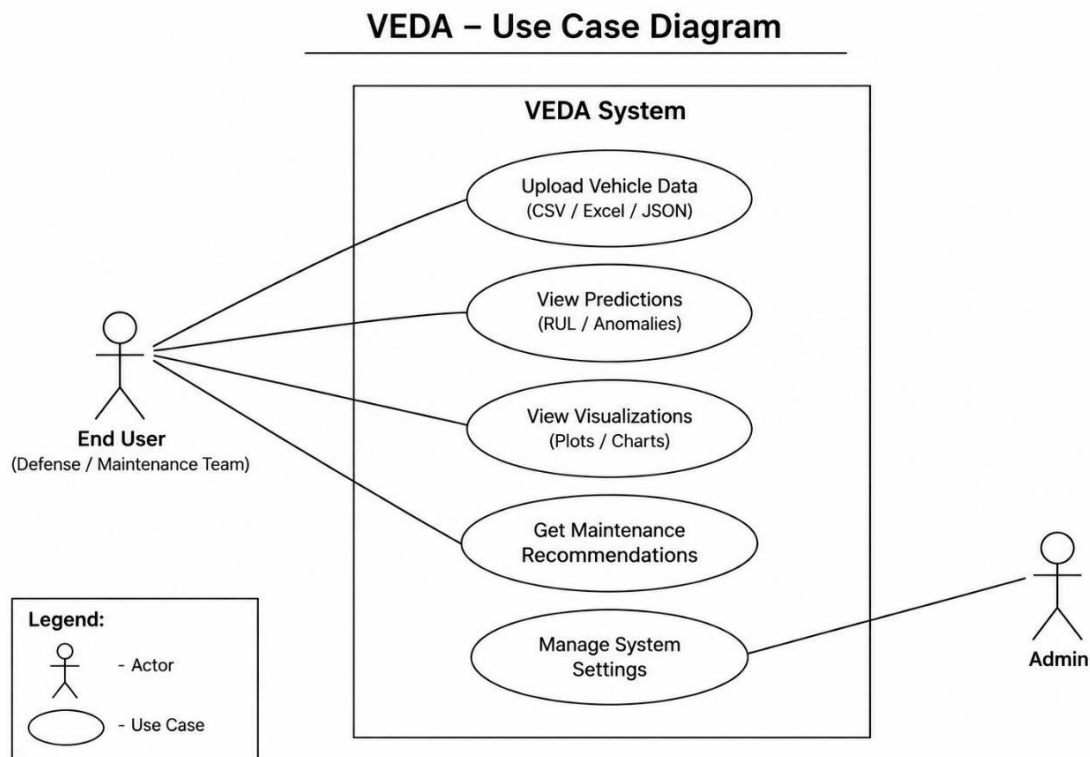


Figure. 5.2.2: Use Case Diagram

5.2.3 Class Diagram

The Class diagram shows the main structure of VEDA's backend and Agentic AI layer. The VedaOrchestrator acts as the central controller, coordinating the six specialized agents, while the MLInferenceService handles model inference and RUL fusion. Pydantic schema classes define the data exchanged between these components and help maintain consistent data flow.

VEDA – Class Diagram

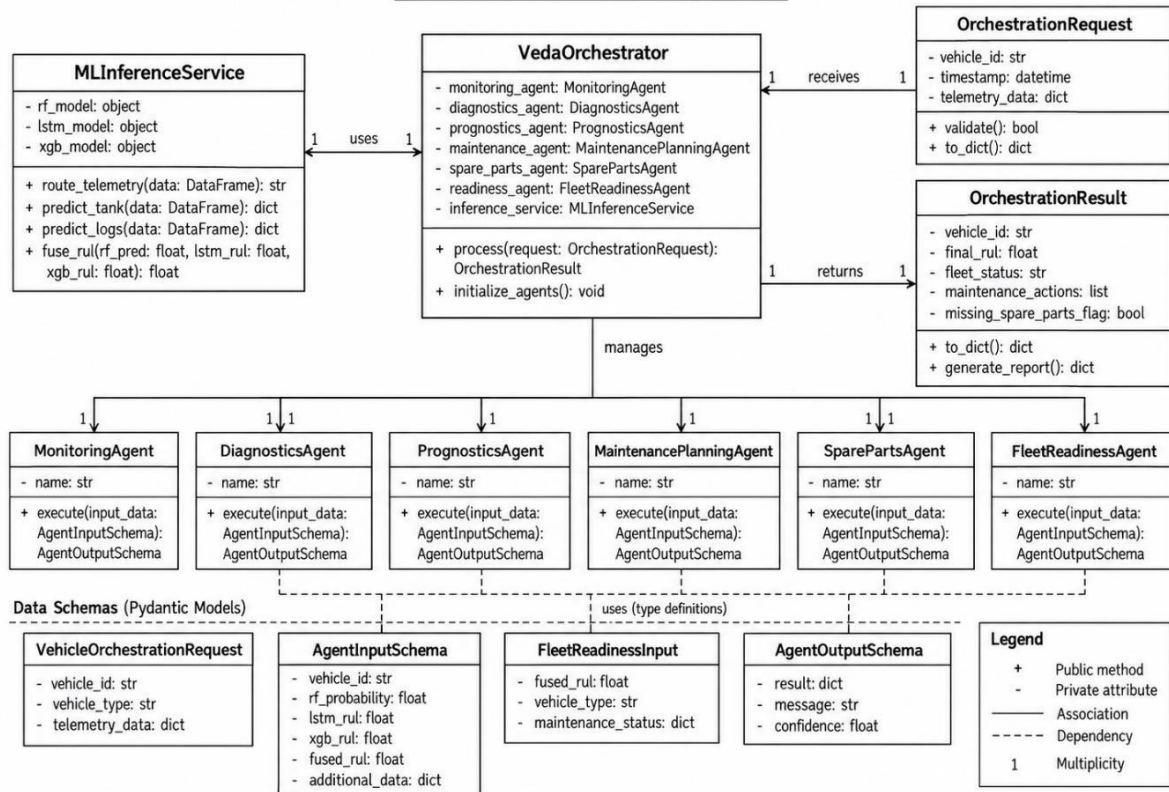


Figure 5.2.3: Class Diagram

5.2.4 Sequence Diagram

The Sequence diagram shows the step-by-step flow of data through VEDA, starting from telemetry upload and ending with the final dashboard response. It shows how the data moves through preprocessing, ML models, RUL fusion, six-agent orchestration, and database storage before the processed result are returned to the user.

VEDA – Sequence Diagram

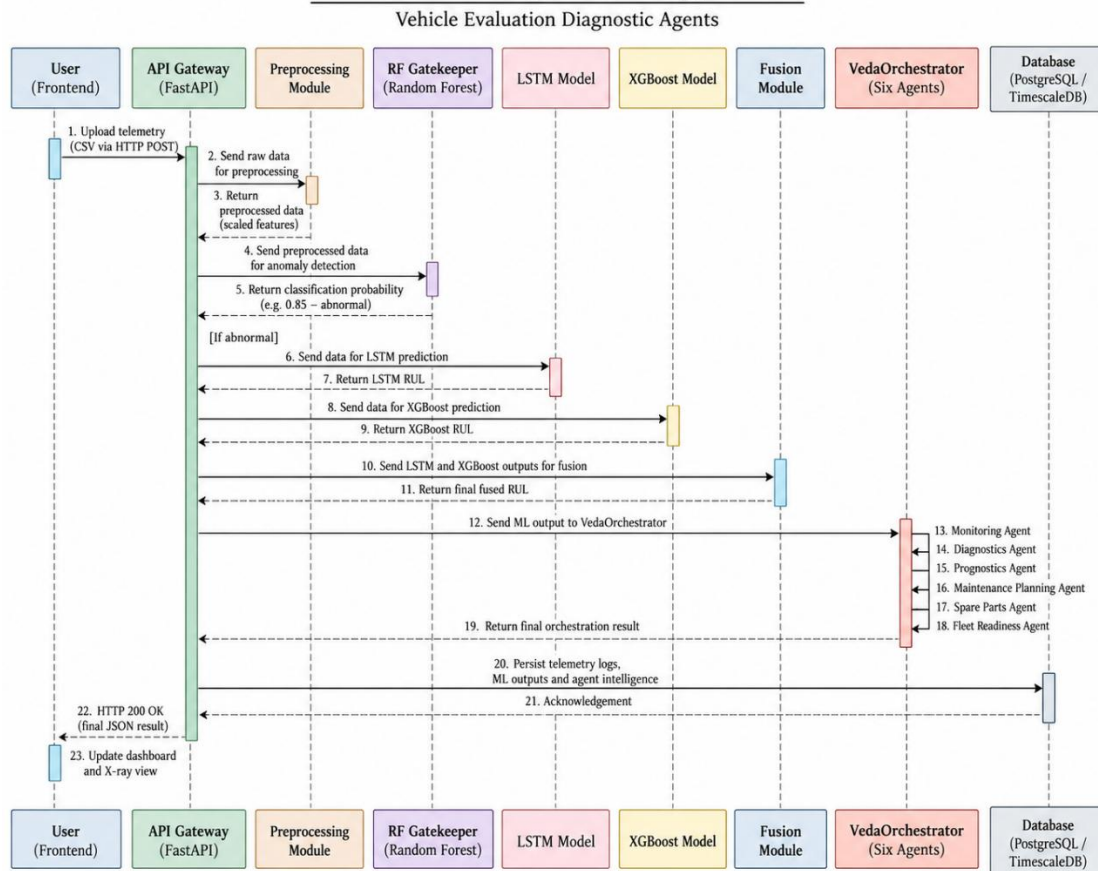


Figure 5.2.4: Sequence Diagram

5.2.5 Activity Diagram

The Activity diagram shows how VEDA processes vehicle telemetry and makes operational decisions. It covers preprocessing, abnormal-condition detection, RUL prediction and fusion, fleet readiness evaluation, spare-parts handling, and finally storing and displaying the results through the Command Center.

VEDA - Activity Diagram

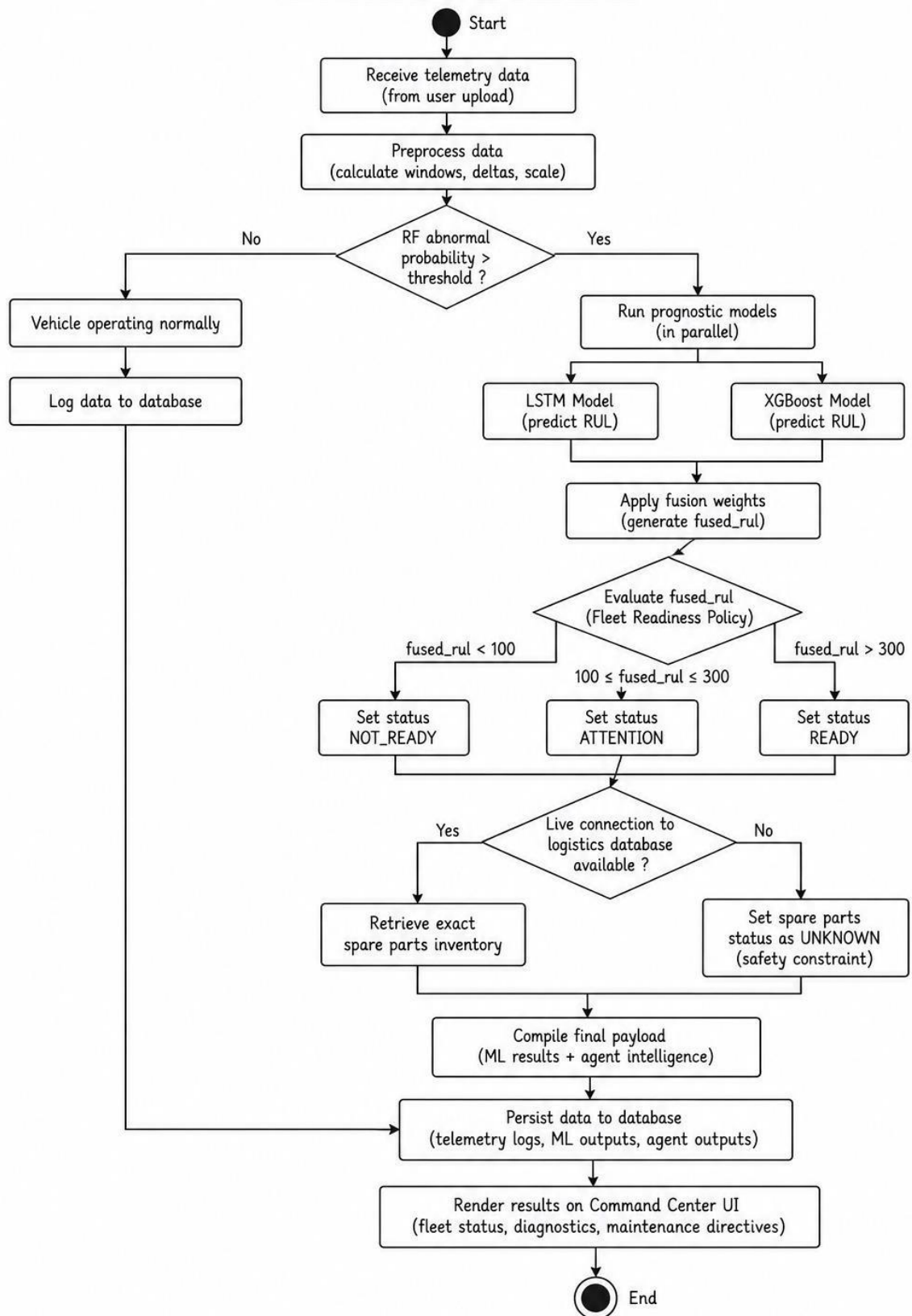


Figure 5.2.5: Activity Diagram — Project Approval to Document Pass

5.2.6 DFD Diagram

The DFD shows how data moves through VEDA, starting from the uploaded vehicle telemetry and passing through preprocessing, classification, RUL prediction, and fusion. The processed results are then handled by the Agentic AI layer to generate vehicle health and readiness information, which is stored in the database and finally presented to the user through the Command Center.

The **Level 0 DFD** provides a high-level view of VEDA, showing the User or Operator providing vehicle telemetry to the system and receiving the final VehicleOrchestrationResponse containing vehicle health and readiness information. The **Level 1 DFD** gives a more detailed view of the internal process, showing how telemetry passes through data normalization, Random Forest classification, prognostic models, RUL fusion, and Agentic AI orchestration before the final results are stored and returned to the user.

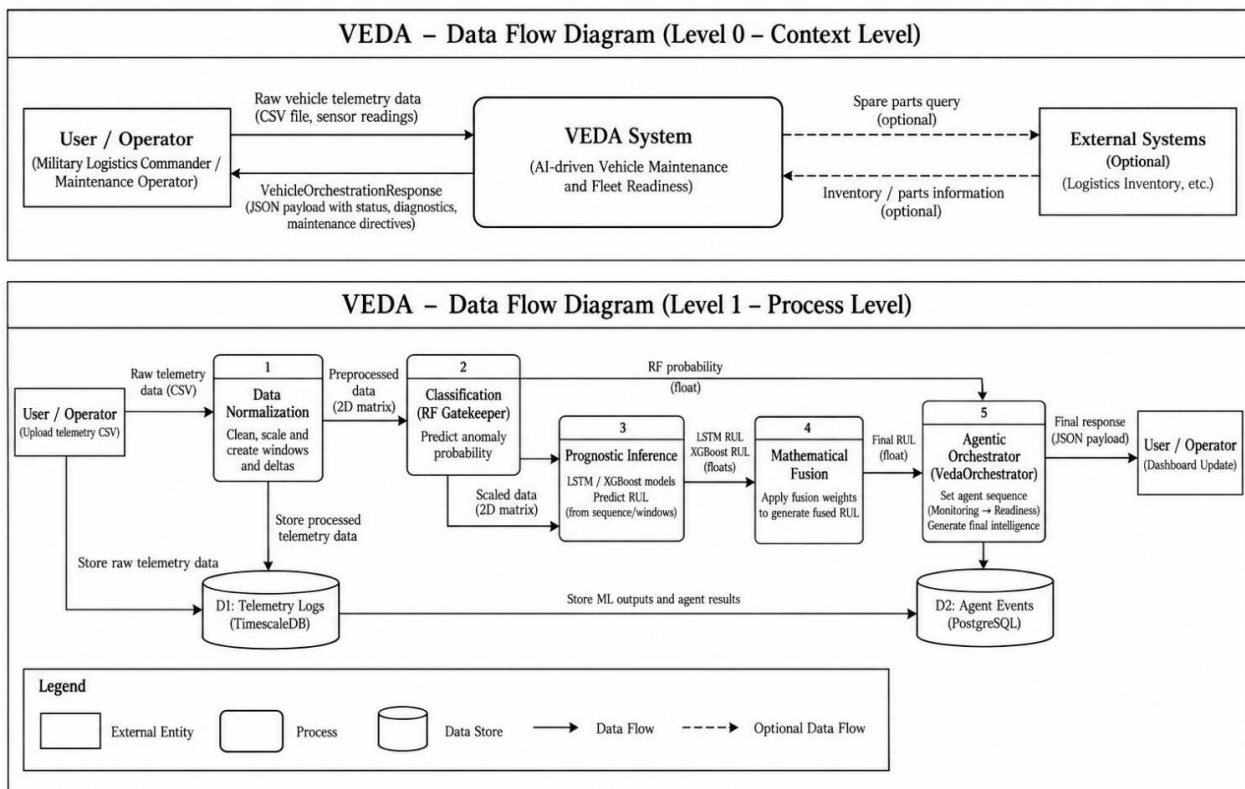


Figure 5.2.6: Data Flow Diagram — Level 0 and Level 1

5.2.7 Deployment Diagram

The deployed system spans three tiers: a client tier (the browser running the React single-page application), an application-server tier hosting the four backend services on BISAG-N infrastructure, and a data tier running MongoDB. All backend services communicate with MongoDB directly rather than through document-service alone, since requirement-service and ai-service each own distinct collections.

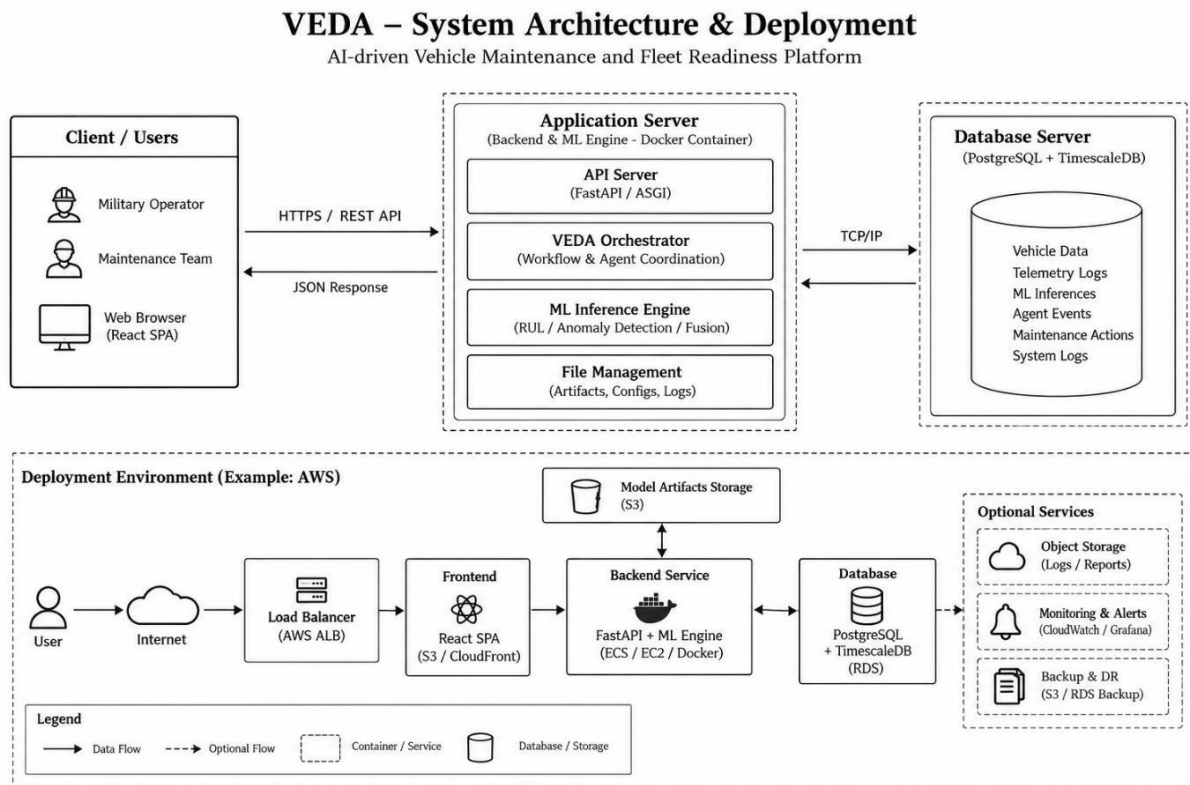


Figure 5.2.7: Deployment Diagram

5.3 Database Design

5.3.1 Table Design & Relationships

PostgreSQL tables with TimescaleDB are used for data storage, while the logical relationships follow the structure shown in the E-R diagram (Section 5.2.1). The primary tables are:

Collection	Key Fields	Related To
vehicles	vehicle_id, type, fleet_id, commissions_date	telemetry_logs, ml_inferences, agent_events
telemetry_logs	log_id, vehicle_id, timestamp, sensor_data	vehicle, ml_inferences
ml_inferences	inferences_id, vehicle_id, telemetry_log_id, rf_probability, lstm_rul, xgb_rul, fused_rul, created_at	vehicles, telemetry_logs, agent_events
agent_events	entry_id, inference_id, diagnostic_flag, maintenance_action, spare_parts_status, fleet_readiness	ml_inferences

Table 5.3.1: PostgreSQL Collections & Relationships

5.3.2 Normalization

Since VEDA uses a **relational PostgreSQL database with TimescaleDB**, the core operational tables follow standard normalization principles up to **3NF**. The high-volume telemetry_logs table uses structured JSONB data where needed to support different vehicle telemetry structures and efficient time-series storage.

Collection	Design Decision	Reason
vehicles	Stores vehicle-specific details separately	Avoids repeating vehicle information across telemetry and inference records
telemetry_logs	Uses a TimescaleDB hypertable with structured JSONB sensor data	Supports high-volume time-series data and different telemetry feature structures
ml_inferences	References vehicle_id and telemetry_log_id using foreign keys	Avoids duplicating vehicle and telemetry information while maintaining traceability
agent_events	References inference_id instead of duplicating ML results	Keeps agent outputs linked to their corresponding inference and reduces data redundancy

Table 5.3.2: Denormalization / Embedding Decisions

5.3.3 Data Dictionary

A representative part of the VEDA data dictionary is provided below, covering the main database tables. The complete dictionary includes the fields used across the vehicles, telemetry_logs, ml_inferences, and agent_events tables described in Section 5.3.1.

vehicles

Field	Type	Constraint	Description
vehicle_id	UUID	Primary Key	Unique identifier for each vehicle
type	VARCHAR	No Null	Identifies the vehicle pipeline: Tank or Logistic_Officer
fleet_id	VARCHAR	Optional	Identifies the fleet or group to which the vehicle belongs

Table 5.3.3.1: Data Dictionary (vehicles)

telemetry_logs

Field	Type	Constraint	Description
log_id	UUID	Primary Key	Unique identifier for the telemetry upload
timestamp	TIMESTAMPTZ	Indexed	Records the time of the sensor reading
sensor_data	JSONB	—	Stores the raw multi-variable vehicle telemetry data

Table 5.3.3.2: Data Dictionary (telemetry_logs)

ml_inferences

Field	Type	Constraint	Description
inference_id	UUID	Primary Key	Unique identifier for the prediction record
rf_probabilty	FLOAT	0.0 to 1.0	Probability of abnormal vehicle behavior
lstm_rul	FLOAT	—	RUL predicted by the LSTM model
xgb_rul	FLOAT	—	RUL predicted by the XGBoost model
fused_rul	INTEGER		Final RUL obtained after model fusion

Table 5.3.3.3: Data Dictionary (ml_inferences)

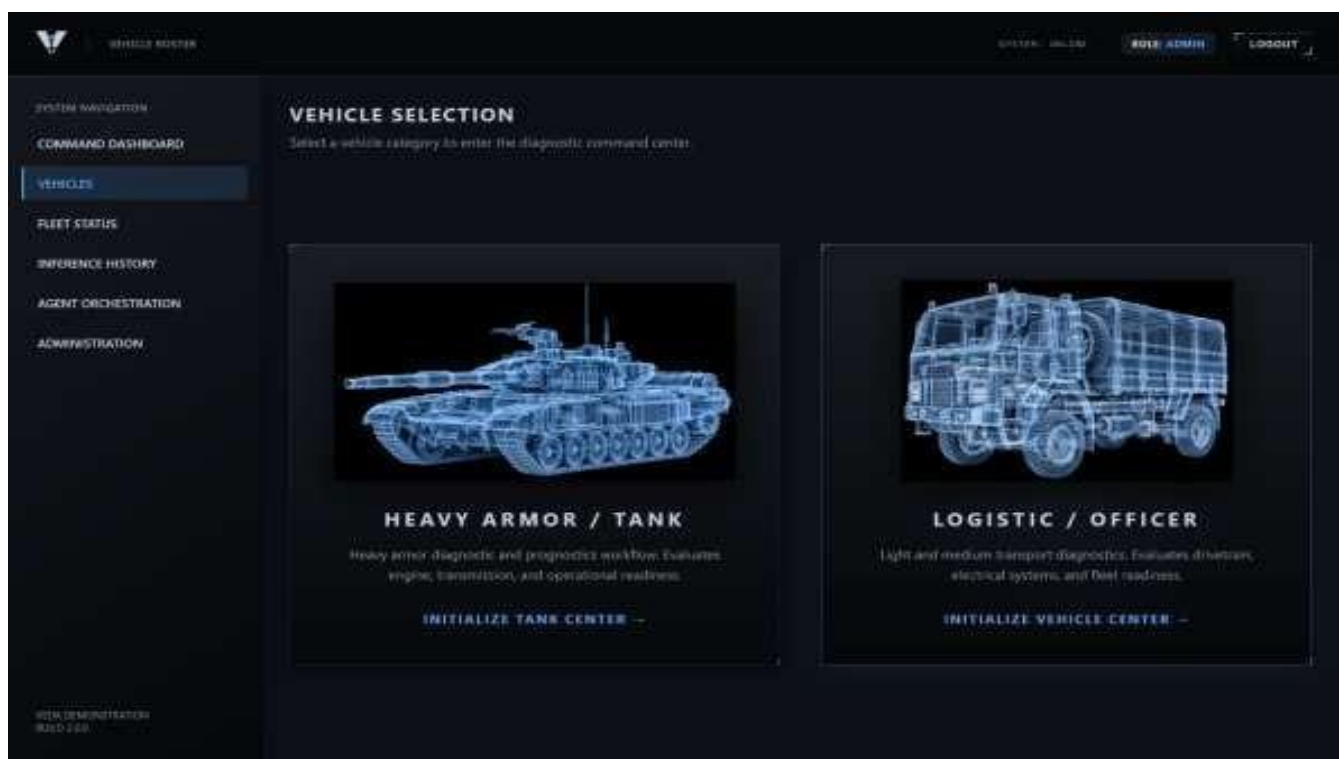
agent_events

Field	Type	Constraint	Description
event_id	UUID	Primary Key	Unique identifier for the agent event
diagnostic-flag	VARCHAR	_	Diagnostic information generated by the Diagnostics Agent
spare_parts_status	VARCHAR	_	Spare-parts availability status, such as UNKNOWN
fleet_readiness	VARChAR	Not Null	Final readiness status such as READY, ATTENTION, or NOT_READY

Table 5.3.3.4: Data Dictionary (agent_events)

5.4 GUI Design

The VEDA GUI is developed using **React and TypeScript** to provide a simple and user-friendly way to view vehicle information. The Command Center allows users to upload telemetry data, monitor the processing, and view the results through vehicle visualizations and information cards. It displays important details such as **RUL, diagnostics, maintenance recommendations, and fleet readiness**, helping users quickly understand the current condition of a vehicle.



Screenshot 5.4.(i): VEDA GUI: Dual-Pipeline Vehicle Selection Dashboard

The **Vehicle Selection interface** provides a professional dashboard for selecting the appropriate vehicle category for diagnostic and predictive maintenance operations. It presents two primary options: **Heavy Armor/Tank**, designed for heavy-armored vehicles with advanced diagnostics and prognostics, and **Logistic/Officer**, intended for light and medium transport vehicles with fleet-level monitoring capabilities. The interface also includes a centralized navigation panel for accessing fleet status, inference history, agent orchestration, and administration modules, enabling streamlined vehicle management and AI-driven diagnostic workflows.

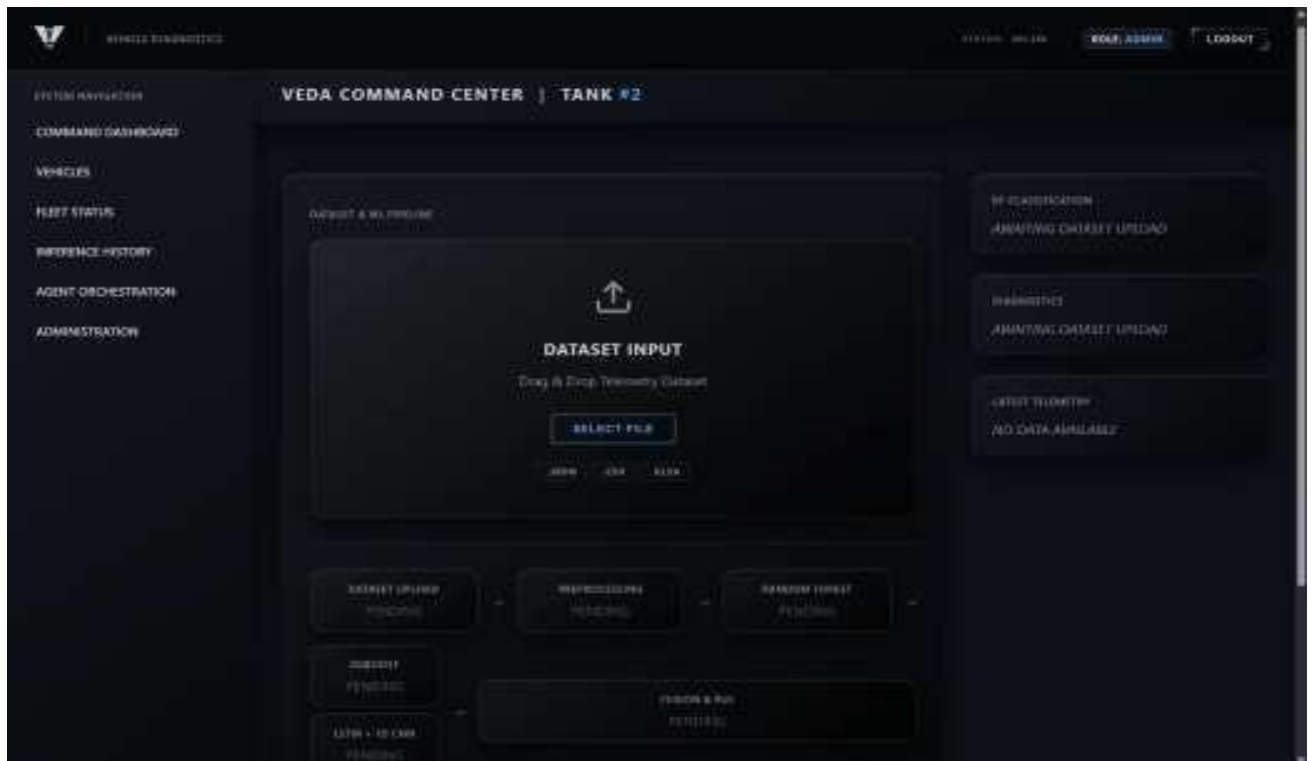
5.5 Screenshots

This section should include screenshots of the running application, at minimum: the project creation wizard, a document review screen showing a score and issue list, the Stage 3 execution dashboard, and the Module C training interface.



Screenshot 5.5(i): VEDA landing page.

The **VEDA welcome screen** serves as the introductory interface of the Vehicle Evaluation and Diagnostic Agent platform. It prominently displays the **VEDA logo and system title**, establishing the platform's identity and purpose, while maintaining a clean and minimal dark-themed design. A central loading indicator suggests that the system is initializing the required diagnostic modules and preparing the user for access to the vehicle evaluation and predictive maintenance environment.



Screenshot 5.5 (ii): VEDA: TANK Command Center

The **VEDA Command Center – Tank #2** provides the primary operational dashboard for initiating and monitoring vehicle diagnostic activities. The interface allows the operator to **upload a vehicle dataset** through the dedicated dataset input panel, with support for structured data such as CSV or similar formats. Alongside the upload section, the dashboard presents key system information, including **diagnostic status, prediction status, and current system state**, enabling the operator to monitor the progress of the analysis. Additional controls and information panels provide access to diagnostic parameters, vehicle metrics, and system outputs, while the left-side navigation allows seamless movement between **Command Dashboard, Vehicles, Fleet Status, Inference History, Agent Orchestration, and Administration** modules. Overall, the interface acts as a centralized control point for initiating AI-based vehicle health assessment and predictive maintenance workflows.



Screenshot 5.5 (iii): VEDA: TANK telemetry data analysis result.

The **Fleet Readiness and Agent Status interface** provides a consolidated view of the vehicle's operational condition and the status of the AI-driven diagnostic agents. At the top, key indicators display the **remaining operational life in hours** and the overall **fleet readiness status**, which is currently shown as ready for operation. The central panel presents the coordinated workflow of multiple specialized agents, including the **Monitoring Agent**, **Diagnostics Agent**, **Prognostics Agent**, **Maintenance Agent**, **Spare Parts Agent**, and **Fleet Readiness Agent**. Each agent is assigned a specific responsibility within the vehicle health management process and is accompanied by a real-time status indicator such as active, available, or not applicable. This interface enables operators to track the progress of individual agents and obtain an integrated view of vehicle health, maintenance requirements, and overall fleet readiness.

DATE	TIMESTEP	RUL (HOURS)	PREDICTION METHOD	READINESS
8/15/2026, 3:49:58 AM	50	395.25	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	HEAVY
8/9/2026, 12:39:17 PM	50	401.83	$0.31 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 12:38:33 PM	50	395.25	$0.31 \times 1574 + 0.70 \times 802\text{hour}$	READY
8/9/2026, 12:09:23 PM	50	376.37	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	READY
8/9/2026, 12:05:52 PM	50	376.37	$0.31 \times 1574 + 0.70 \times 802\text{hour}$	READY
8/9/2026, 12:04:38 PM	50	401.83	$0.31 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:55:19 AM	50	376.37	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	READY
8/9/2026, 11:55:00 AM	50	400.66	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	READY
8/9/2026, 11:53:55 AM	15	UNKNOWN	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	UNKNOWN
8/9/2026, 11:53:54 AM	20	441.73	$0.35 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:53 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:53 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.31 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION
8/9/2026, 11:53:52 AM	30	441.73	$0.30 \times 1574 + 0.70 \times 802\text{hour}$	ATTENTION

Screenshot 5.5 (iv): VEDA: Vehicles Inference History.

The **Predictive Maintenance History** interface provides a detailed record of previous vehicle diagnostic and prognostic analyses performed by the VEDA platform. The table organizes historical inference results using key parameters such as **date**, **timestamp**, **remaining useful life (RUL)**, **prediction method**, and **readiness status**. Each entry represents an individual analysis cycle, allowing operators to review the predicted operational life of the vehicle and the corresponding system assessment. The **Readiness** column categorizes the vehicle condition using status indicators such as **READY**, **ATTENTION**, and **UNAVAILABLE**, making it easier to identify historical periods requiring monitoring or maintenance intervention. By maintaining a chronological record of inference results, this module supports **trend analysis**, **maintenance planning**, **performance tracking**, and **data-driven fleet management**.

6 Implementation Planning

VEDA was implemented using a structured development environment with Python virtual environments for ML development, FastAPI and Pydantic for the backend, PostgreSQL with TimescaleDB for data storage, and React with TypeScript for the frontend. Scikit-learn, TensorFlow/Keras, XGBoost, Pandas, and NumPy were used for the ML pipeline and data processing. Standard coding practices, data validation, and error-handling mechanisms were followed to ensure consistent and reliable system operation.

6.1 Implementation Environment

Category	Detail
Operating Systems	macOS / Windows 11 / Ubuntu 22.04
Development Hardware	Multi-core processor, 16 GB RAM
IDE	Visual Studio Code
Backend Runtime	Python 3.10+, FastAPI
AI Service Runtime	Scikit-learn, TensorFlow/Keras, XGBoost
Database	PostgreSQL with TimescaleDB
Frontend	React, TypeScript, Vite
Version Control	Git for maintaining and tracking the codebase

Table 6.1: Implementation Environment

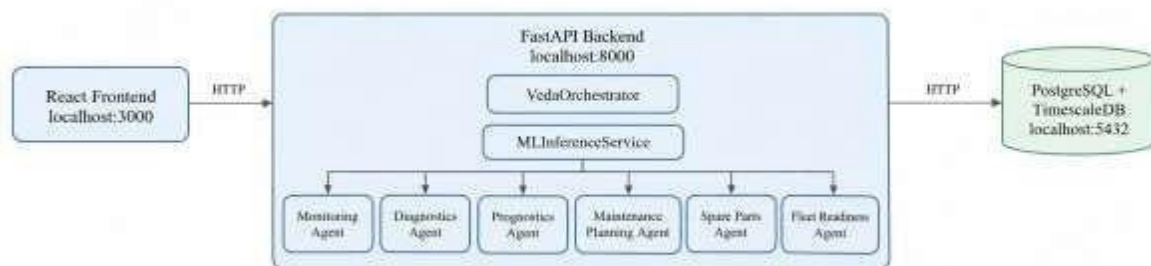


Figure 6.1: Local Development Setup / Service Ports Diagram

6.2 Tools & Technologies Used

The following table summarizes the main tools and technologies used in VEDA and their roles in the project.

Layer / Purpose	Tool / Technology	Role in the Project
Frontend framework	React, TypeScript	Used to build the Command Center interface and display vehicle health and prediction results.
Frontend build tool	vite	Used for frontend development and fast application updates.
Frontend Animation	Motion.dev	Used to add animations and interactive visual elements to the interface.
Backend Framework	FastAPI	Provides the backend APIs and connects the frontend with the ML and Agentic AI components.
Data Validation	Pydantic	Validates API data and maintains consistent data structures between system components.
Machine Learning	Scikit-Learn	Used for the Random Forest models for abnormal-condition classification.
Deep Learning	TensorFlow/Keras	Used to develop and run the LSTM and 1D-CNN models for RUL prediction.
RUL prediction	XGBoost, LSTM+1D CNN	Used as an additional prognostic model for RUL prediction.
Data Preprocessing	Pandas, NumPY	Used for telemetry processing, feature engineering, and preparation of model inputs.
Database	PostgreSQL, TimescaleDB	Used to store vehicle, telemetry, ML inference, and agent-related data.

Database ORM	SQLAlchemy	Used to interact with the PostgreSQL database from the backend.
API communication	REST over HTTPS	Communication protocol between the frontend, api-gateway, and backend services
Development	Visual Studio Code, Git	Used for coding, project management, and version control.

Table 6.2: Tools & Technologies Used

6.3 Coding Standards Followed

The following coding standards were followed during the development of VEDA:

- **PEP 8 Guidelines:** Python code was written using standard formatting and naming conventions to keep the code clean and readable.
- **Type Hints:** Python type hints were used where required to make functions and data structures easier to understand and maintain.
- **Data Validation:** Pydantic schemas were used to validate API requests and responses and maintain consistent data between system components.
- **Modular Development:** ML models, backend services, agents, and database operations were kept as separate modules for easier maintenance and testing.
- **Error Handling:** try-except blocks and defined fallback values such as UNKNOWN were used to handle errors and missing information safely.
- **Version Control:** Git was used to track code changes and maintain different development stages of the project.

6.3.1 Project & Folder Structure

Folder	Purpose
backend/app/	Contains the main FastAPI backend application.
backend/app/agents/	Contains the six Agentic AI modules, with separate logic and schemas for each agent
backend/app/api/	Contains the FastAPI API routes and endpoints.
backend/app/models/	Contains SQLAlchemy models used for database operations.
backend/app/services/	Contains the MLInferenceService for loading frozen ML models and generating predictions and fusion results.
frontend/	Contains the React and TypeScript code for the Command Center interface.
ml/	Stores the trained and frozen machine learning models, separated by vehicle category.
scripts/	Contains scripts used for dataset validation, model testing, leakage checks, and other development tasks.
datasets/	Stores the telemetry datasets used for model training, validation, testing, and system testing.

Table 6.3.1: Project & Folder Structure

6.3.2 Naming & Formatting Conventions

Area	Convention
Pydantic schema classes	PascalCase with clear names such as VehicleOrchestrationRequest and FleetReadinessInput
Frozen ML artifacts	snake_case with algorithm, vehicle category, and version, such as rf_tank_v1.pkl and lstm_logistic_v2.h5
Database tables and columns	snake_case, such as telemetry_logs and fused_rul
Agent status outputs	Capitalized fixed values such as READY, ATTENTION, and UNKNOWN
Python functions and variables	snake_case following standard PEP 8 conventions
React/TypeScript components	PascalCase
REST API endpoints	Lowercase, hyphenated resource names

Table 6.3.2: Naming & Formatting Convention

6.3.3 Error Handling & API Responses

VEDA uses a structured error-handling approach to keep the system reliable and prevent unsupported information from being generated. If an agent receives invalid input or an external dependency is unavailable, the system uses proper exception handling and returns safe fallback values such as UNKNOWN instead of stopping the complete process. The FastAPI backend provides standardized JSON responses, with successful inference requests returning a 200 OK response. If the uploaded telemetry data does not contain the required features, validation stops the inference process and returns a 422 Unprocessable Entity response with details about the missing information.

6.3.4 Documentation & Comments

The VEDA project follows a clear documentation approach to make the code easier to understand and maintain. Python docstrings are used for important classes and functions in the backend, ML inference, and Agentic AI modules to describe their inputs, outputs, and purpose. Inline comments are added to complex parts of the code, such as feature engineering and data preprocessing, to explain the required transformations. The project also includes Markdown documentation covering the system architecture, Agentic AI orchestration, decision boundaries, and validation activities, providing developers with a clear reference for understanding and maintaining the system.

7 Limitations and Future Scope

No system of this scope is delivered without trade-offs and open issues. This section records the limitations known at the current stage of development including issues still being actively tracked alongside directions identified for extending the system beyond its present scope.

7.1 Limitations

Limitation	Description
Synthetic Data Dependency	The Tank and Logistic/Officer ML pipelines are trained and validated using engineered synthetic telemetry. The models have not yet been tested with real military vehicle sensor data and its associated noise and variations.
Limited Spare Parts Information	VEDA is not connected to a live military ERP or inventory system. Therefore, the Spare Parts Agent cannot provide live inventory information and uses defined fallback values such as UNKNOWN when the required information is unavailable.
Batch Data Processing	The current system processes telemetry through batch file uploads. Real-time continuous data streaming from vehicle sensors or CAN bus is not currently implemented.

Table 8.1: Limitations Summary

7.2 Future Scope

- **New Vehicle Pipelines:** Extend VEDA with separate ML pipelines for additional vehicle categories such as UAVs and other military platforms.
- **Live Inventory Integration:** Connect the Spare Parts Agent with an authorized inventory or ERP system to provide up-to-date spare-parts information.
- **Edge Deployment:** Deploy the ML inference system on edge devices such as NVIDIA Jetson to support local and offline vehicle health monitoring.

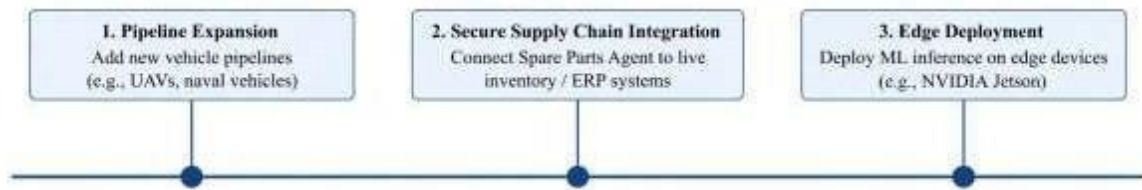


Figure 8.2: Future Scope Roadmap (Indicative Priority Order)

Planned Enhancement	Motivation
Support for New Vehicle Pipelines	New isolated ML pipelines and feature contracts can be added to support other vehicle categories such as UAVs or naval vehicles.
Live Supply Chain Integration	Connecting the Spare Parts Agent with an authorized inventory system would allow it to use current stock information and support maintenance-related parts planning.
Edge Deployment	Deploying the ML inference system on edge devices such as NVIDIA Jetson could enable local vehicle health monitoring and diagnostics without depending on continuous network connectivity.

Table 8.2: Future Scope Summary

8 Conclusion and References

8.1 Conclusion

The VEDA (Vehicle Evaluation & Diagnostics Agent) project demonstrates an automated approach to vehicle health monitoring and predictive maintenance for military ground vehicles. The system uses separate machine learning pipelines for Tank and Logistic/Officer vehicles, helping maintain clear separation between their data, features, and models. LSTM + 1D-CNN and XGBoost models are used for RUL prediction, with their results combined through predefined fusion configurations.

The Agentic AI layer further processes these results through six specialized agents for monitoring, diagnostics, prognostics, maintenance planning, spare parts, and fleet readiness. Defined rules and fallback conditions help the system provide consistent outputs when information is missing or unavailable. The FastAPI backend connects the ML and Agentic AI components, while the React-based Command Center provides a simple interface for viewing vehicle health, RUL, maintenance information, and fleet readiness. Overall, VEDA provides a modular foundation that can be extended with additional vehicle pipelines, live inventory integration, and edge deployment in the future.

8.2 References

Machine Learning and Data Science

- Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. <https://scikit-learn.org/stable/>
- Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. <https://xgboost.readthedocs.io/>
- Abadi, M., et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. <https://www.tensorflow.org/>

Backend and Database

- Ramírez, S. (2020). *FastAPI Documentation*. FastAPI Documentation. <https://fastapi.tiangolo.com/>
- TimescaleDB Documentation. *Time-Series Data Management on PostgreSQL*. <https://docs.timescale.com/>

Frontend Development

- React Documentation. *A JavaScript Library for Building User Interfaces*. <https://reactjs.org/>
- Motion.dev Documentation. *Animation Library for React*. <https://motion.dev/examples/react-scroll-horizontal>

9 Appendices

This section collects supplementary reference material installation steps, configuration reference, and consolidated lists that supports the main chapters without interrupting their flow.

Appendix A Installation & Setup Guide

The following steps are used to run the VEDA system locally, consistent with the implementation environment described in Section 6.1.

- Install the required prerequisites: **Python 3.9+**, **Node.js v18+**, **PostgreSQL**, and **TimescaleDB**.
- Clone the VEDA repository and install the backend dependencies using the Python requirements file.
- Configure the PostgreSQL database and enable the TimescaleDB extension.
- Start the FastAPI backend so that the ML inference and Agentic AI components are available.
- Install the frontend dependencies using Node.js and start the React Command Center in development mode.
- Confirm that the frontend can communicate with the FastAPI backend and that the database connection is working correctly.

Exact Startup Commands & Order:

1. **Install Backend Dependencies:** Navigate to the backend/ directory and create the virtual environment:

```
cd backend
```

```
python -m venv venv
```

Activate the environment and install the required packages:

```
pip install -r requirements.txt
```

2. **Start Backend Services (in separate terminal windows):**

Start the FastAPI Server:

```
uvicorn app.main:app --reload --port 8000
```

3. Start Frontend:

Navigate to the frontend directory and install the required packages:

```
cd frontend
```

```
npm install
```

Start the React Command Center:

```
Npm run dev
```

4. Database Setup:

Ensure that **PostgreSQL is running on port 5432** and that the **TimescaleDB extension** is enabled for the VEDA database.

Appendix B: Environment Variables Reference

Variable	Description
DATABASE_URL	PostgreSQL database connection string.
ML_ARTIFACT_PATH	Path to the /ml directory containing the frozen ML models.
FRONTEND_CORS_ORIGIN	Defines the frontend origin allowed to access the backend API.
FLEET_POLICY_VERSION	Specifies the Agentic AI fleet-readiness ruleset to use, such as demo_v1.

Table B.1: Appendix B: Environment Variables Reference

Appendix C: Supported Vehicle Models and Pipelines

Consolidated from Section 2.1 of the user manual and Section 1.5 — the seven SDLC models and the rules the system enforces for each.

Vehicle Category	Pipeline	Input	ML Models	Fusion
Heavy Armor (Tank) 1	Module A	43 features	RF, LSTM + 1D-CNN, XGBoost	0.30 / 0.70
Logistic / Officer	Module B V2	28 features	RF V2, LSTM V2, XGBoost V2	0.30 / 0.70

Table C.1: Appendix C: Supported Vehicle models and pipelines

Appendix D: Full Agent Action List

Agent	Main Action
Monitoring Agent	Processes telemetry information and checks for missing sensor data.
Diagnostics Agent	Uses abnormal-condition information to identify affected vehicle subsystems.
Prognostics Agent	Processes the LSTM and XGBoost outputs and provides the fused RUL result.
Maintenance Planning Agent	Provides maintenance actions based on the available diagnostic information.
Spare Parts Agent	Checks spare-parts information and returns UNKNOWN when the required inventory information is unavailable.
Fleet Readiness Agent	Applies the demo_v1 ruleset to the available vehicle information and determines the fleet-readiness status.

Table D.1: Full Agent Action List

Appendix E: System Architecture and API Reference

This appendix provides a consolidated reference for the software architecture, communication flow, and major API interfaces implemented within the VEDA system. The architecture follows a modular design in which the frontend, backend, database, machine-learning models, and Agentic AI components operate as separate but interconnected layers. This separation allows individual components to be maintained, tested, and extended without significantly affecting the remaining system.

E.1 System Layer Architecture

The VEDA platform is organized into the following major layers:

1. **Presentation Layer** – Provides the React-based Command Center through which operators interact with the system.
2. **Application Layer** – Implemented using FastAPI and responsible for request handling, validation, authentication-related processing, and orchestration.
3. **Machine Learning Layer** – Contains the frozen machine-learning models responsible for vehicle health prediction and Remaining Useful Life (RUL) estimation.
4. **Agentic Intelligence Layer** – Coordinates specialized agents for monitoring, diagnostics, prognostics, maintenance planning, spare-parts assessment, and fleet readiness.
5. **Data Layer** – Uses PostgreSQL and TimescaleDB for structured storage of vehicle information, telemetry, inference history, and time-series information.
6. **Reporting and Monitoring Layer** – Converts model and agent outputs into operator-readable diagnostic and fleet-readiness information.

The layered architecture ensures that the user interface does not directly communicate with machine-learning models or database components. All major operations are routed through the backend API, thereby providing a controlled and consistent communication mechanism.

E.2 Frontend–Backend Communication

The React Command Center communicates with the FastAPI backend through HTTP-based REST API requests. When an operator uploads a dataset or requests an inference operation, the frontend sends the corresponding request to the backend.

The general communication sequence is:

Operator → React Command Center → FastAPI API → Data Validation → ML Inference → Agent Orchestration → Database → API Response → Command Center

The backend validates the received information before passing it to the appropriate processing module. This prevents invalid or incomplete data from directly reaching the machine-learning pipeline.

API Function	Purpose	Primary Output
Vehicle Selection	Selects the vehicle category	Selected vehicle type
Dataset Upload	Accepts vehicle telemetry data	Dataset validation status
Inference Request	Initiates ML prediction	Prediction results
Diagnostic Request	Performs vehicle condition assessment	Diagnostic information
RUL Prediction	Estimates remaining useful life	RUL value
Agent Execution	Executes the appropriate agents	Agent status and outputs
Fleet Readiness	Determines operational readiness	Readiness status
Inference History	Retrieves previous analyses	Historical inference records
Vehicle Status	Displays current vehicle condition	Current health status

Table E.1: Frontend–Backend Communication

E.4 Request Processing Workflow

A typical inference request follows a controlled sequence. First, the operator selects the appropriate vehicle category and provides the required dataset. The backend checks whether the uploaded file conforms to the expected structure and whether the required features are available.

After successful validation, the relevant machine-learning pipeline is selected. The model generates the required prediction, after which the result is passed to the Agentic AI layer. The agents process the prediction together with diagnostic and operational information and generate a structured vehicle assessment.

The final response is returned to the frontend and displayed through the Command Center.

E.5 Error Handling

The system incorporates validation and error-handling mechanisms to prevent invalid requests from disrupting the diagnostic workflow. Typical validation conditions include:

- Missing dataset.
- Unsupported file format.
- Missing required features.
- Invalid numerical values.
- Incorrect vehicle category.
- Unavailable machine-learning model.
- Database connectivity failure.
- Invalid API request.
- Incomplete inference result.

When an error occurs, the backend returns an appropriate response to the frontend instead of exposing internal implementation details. This allows the Command Center to display an understandable status message to the operator.

Appendix F: Machine Learning Inference and Data Processing Reference

This appendix consolidates the data-processing and machine-learning workflow used by the VEDA platform. The purpose of this reference is to provide a clear relationship between the input telemetry, preprocessing operations, machine-learning models, prediction outputs, and final vehicle assessment.

F.1 Input Telemetry

The VEDA system operates on structured vehicle telemetry containing operational and sensor-related information. Depending on the selected vehicle category, the input dataset contains the features required by the corresponding diagnostic and prognostic pipeline.

The input data may include:

- Vehicle identification information.
- Operational cycle information.
- Operating-condition parameters.
- Engine or subsystem sensor measurements.
- Historical telemetry values.
- Model-specific diagnostic parameters.

The data is processed according to the selected vehicle pipeline rather than applying a single generic model to all vehicle categories.

F.2 Data Validation

Before inference, the backend performs a validation process to verify the structure and quality of the incoming data.

The validation process checks:

Dataset Received → File Validation → Column Validation → Data-Type Validation → Missing-Value Check → Feature Preparation → Model Inference

This process ensures that the ML models receive data in the expected format.

Where sensor values are missing, the system identifies the missing information and provides the corresponding status to the Monitoring Agent. This prevents missing information from being silently interpreted as a valid sensor reading.

F.3 Feature Preparation

Feature preparation converts raw telemetry into the representation expected by the machine-learning models. The preprocessing stage may include:

- Selection of relevant features.
- Numerical conversion.
- Missing-value handling.
- Feature scaling where required.
- Temporal ordering of observations.
- Window-based processing.
- Construction of model-specific input arrays.

The preprocessing configuration is maintained separately from the inference logic to ensure that the same transformation procedure can be consistently applied during model execution.

F.4 Machine-Learning Model Execution

VEDA uses multiple machine-learning models within its supported diagnostic and prognostic pipelines. These include **Random Forest (RF)**, **Long Short-Term Memory (LSTM)**, **1D Convolutional Neural Network (1D-CNN)**, and **XGBoost** models.

The models have complementary characteristics. Tree-based models such as Random Forest and XGBoost can capture nonlinear relationships between sensor variables, while LSTM models are suitable for sequential telemetry patterns. The 1D-CNN component can extract local patterns from structured sensor sequences.

The system therefore does not depend exclusively on a single prediction model.

F.5 Prediction Fusion

For supported pipelines, model outputs are combined using a predefined fusion mechanism. The documented fusion configuration uses:

RF Contribution = 0.30

Prognostic Model Contribution = 0.70

The resulting fused value is used as the primary prognostic output for subsequent Agentic AI processing.

The fusion mechanism provides a consistent approach for combining predictions from different models while maintaining a deterministic configuration.

F.6 Remaining Useful Life Estimation

Remaining Useful Life (RUL) represents the estimated amount of operational life remaining before a defined degradation or maintenance condition is reached.

The general workflow is:

Telemetry → Feature Processing → ML Models → Prediction Fusion → RUL Estimate → Prognostics Agent → Fleet Readiness Assessment

The RUL prediction is not interpreted independently. The Prognostics Agent combines the available prognostic information with other system information before passing the result to the downstream maintenance and readiness components.

F.7 Model Output Structure

Field	Description
Vehicle ID	Identifier of the analyzed vehicle
Vehicle Category	Selected vehicle classification
Inference Timestamp	Time at which inference was performed
RUL Prediction	Estimated remaining useful life
Diagnostic Status	Current diagnostic assessment
Model Status	Status of the ML inference process
Agent Status	Status of the relevant AI agents
Maintenance Recommendation	Generated maintenance information
Fleet Readiness	Final operational readiness state

Table F.1: Model Output Structure

F.8 Inference History

Every completed inference can be represented as a historical record containing the timestamp, prediction, prediction method, and resulting readiness state. These records allow operators to review previous analyses and observe changes in vehicle condition over time.

Historical inference information is particularly useful for identifying recurring diagnostic conditions and supporting maintenance planning. Instead of treating each prediction as an isolated event, the system provides a chronological record of vehicle assessments.

Report Verification Procedure

Date:

Project Name: VEDA (Vehicle Evaluation Diagnostics Agent): Enables Proactive Vehicle Maintenance by Predicting Failures and Optimizing Fleet Readiness Using AI-Driven Analytics

Student Name & ID:

- 1. Adhish Prasad, ID: 26BISAG0542**
- 2. Harsh Barbhaya, ID: 26BISAG0543**
- 3. Yash Verma, ID: 26BISAG0544**

	Soft Copy	Hard Copy
Report Format:		
Project Index:		

Sign by Training Coordinator

Sign by Project Guide